

计算机网络基础

OSI七层协议

内核公共：物理层，数据链路层，网络层，运输层，

用户空间：会话层，表示层，应用层

物理层：主要解决两台物理机之间的通信，通过**二进制比特流的传输**来实现，二进制数据表现为电流电压上的强弱，到达目的地再转化为二进制机器码。（网卡、集线器）

数据链路层：接收来自物理层的位流形式的数据，并**封装成帧**，传送到上一层；同样，也将来自上层的数据帧，拆装为位流形式的数据转发到物理层。这一层在物理层提供的比特流的基础上，通过差错控制、流量控制方法，使有差错的物理线路变为无差错的数据链路。提供物理地址寻址功能。（交换机）

网络层：**将网络地址翻译成对应的物理地址**，并决定如何=将数据从发送方路由到接收方=，通过路由选择算法为分组通过通信子网选择最佳路径。（路由器）（IP）（下一跳机制）

运输层：运输层**提供了进程间的逻辑通信**，运输层向高层用户屏蔽了下面网络层的核心细节，使应用程序看起来像是在两个传输层实体之间有一条端到端的逻辑通信信道（TCP，UDP）

会话层：

建立会话：身份验证，权限鉴定等；

保持会话：对该会话进行维护，在会话维持期间两者可以随时使用这条会话传输局；

断开会话：当应用程序或应用层规定的超时时间到期后，OSI会话层才会释放这条会话。

表示层：对数据格式进行编译，**对收到或发出的数据根据应用层的特征进行处理**，如处理为文字、图片、音频、视频、文档等，还可以对压缩文件进行解压缩、对加密文件进行解密等。

应用层：**提供应用层协议**，如HTTP协议，FTP协议等等，方便应用程序之间进行通信。

数字证书

私钥用来做数字签名，具体分三个步骤：

- (1)对要签名的数据，用公开的哈希算法计算出哈希值，这个哈希值也叫摘要。
- (2)用CA私钥对摘要加密，得到的密文就是私钥拥有者的“数字签名”。
- (3)将数据与“数字签名”放在一起，发送给接收者。

接收者收到数据后的认证逻辑：

- (1)用CA公钥解密“数字签名”，得到发送方给出的数据摘要。
- (2)用发送方采用的哈希算法，自己计算出数据摘要。
- (3)如果发送方给出的摘要和自己计算出的摘要相等，证明数据有效。

假如有一个窃取者，中途截获了数据，并偷偷做了两件事情：

1. 偷偷将接收者手上持有的公钥替换成自己的公钥。
2. 篡改数据，然后用自己的私钥对数据重新签名。

该问题产生的原因是：**接收者不能确认自己持有的公钥是不是发送者的。**

数字证书的制作过程：

CA机构准备好自己的私钥和公钥，并且**向外界提供公开的公钥**获取途径。

需要数字证书的发送者，拿着自己公开的基本信息和公钥，前往CA机构要求认证。

CA机构用自己的私钥对发送者基本信息和公钥一起加密，得到的密文就是数字证书，交给发送者带回去。

发送者有权威机构颁发的数字证书以后，就可以通过数字证书把自己的公钥交给接收者，逻辑如下：

发送者把数据、数字签名、数字证书三样东西一并发送给接收者。

接收者通过CA机构提供的公开途径，获取到CA的公钥。

接收者收到数据以后，用**CA的公钥解密数字证书**，拿到证书里包含的**基本信息和公钥**。

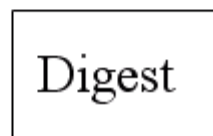
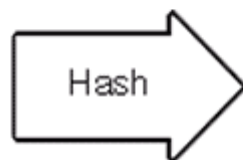
接收者**对比基本信息**，如果和发送者的一致，则证明公钥是发送者的；如果信息不一致，则说明公钥不是发送者的。

如此这般，接收者认证发送者数据的过程就变成了：

先通过CA公钥认证数字证书，确定**发送者公钥的正确性**。

再通过发送者公钥认证数字签名，确定**数据内容的有效性**。

摘要



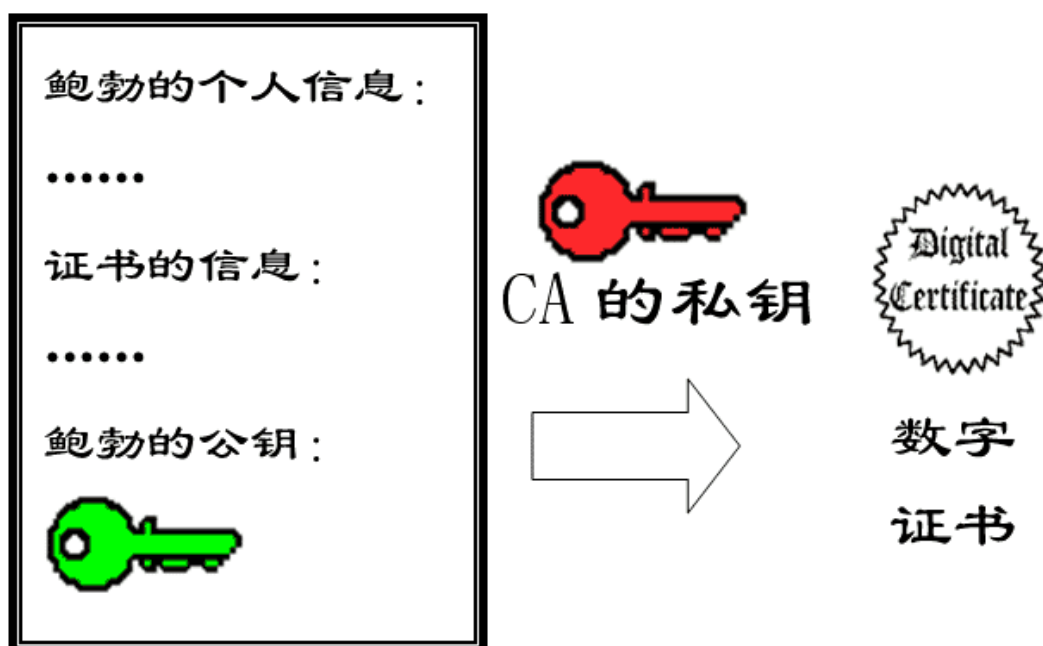
<http://blog.csdn.net/hp910315>

数字签名



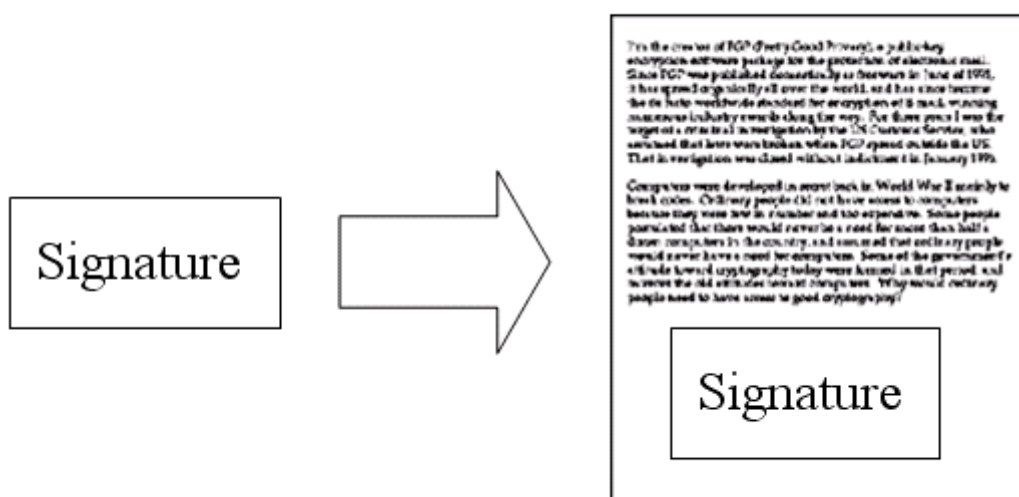
<http://blog.csdn.net/hp910315>

数字证书 (确保发送者公钥的正确性, 即身份的正确性)

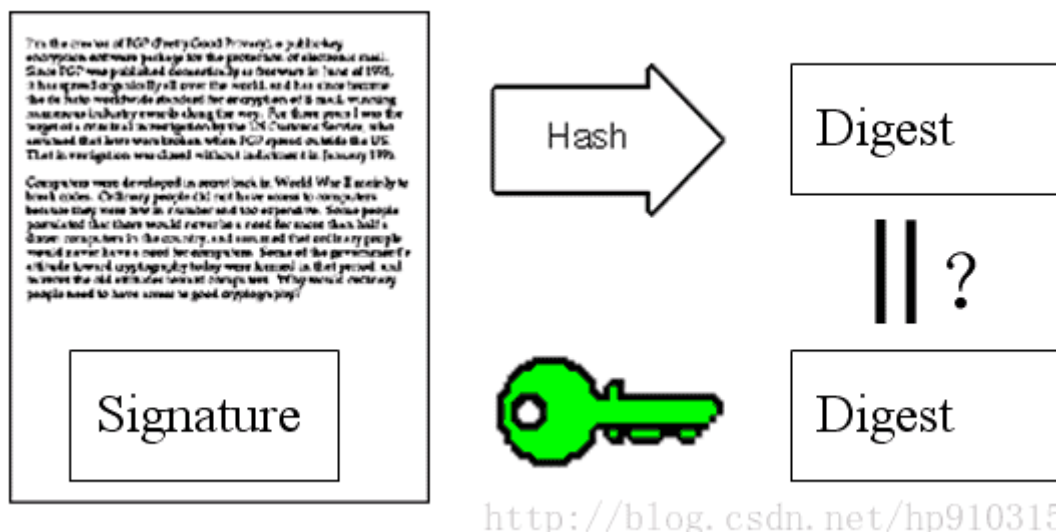


<http://blog.csdn.net/hp910315>

(1) 公钥被替换, 无数字证书



<http://blog.csdn.net/hp910315>

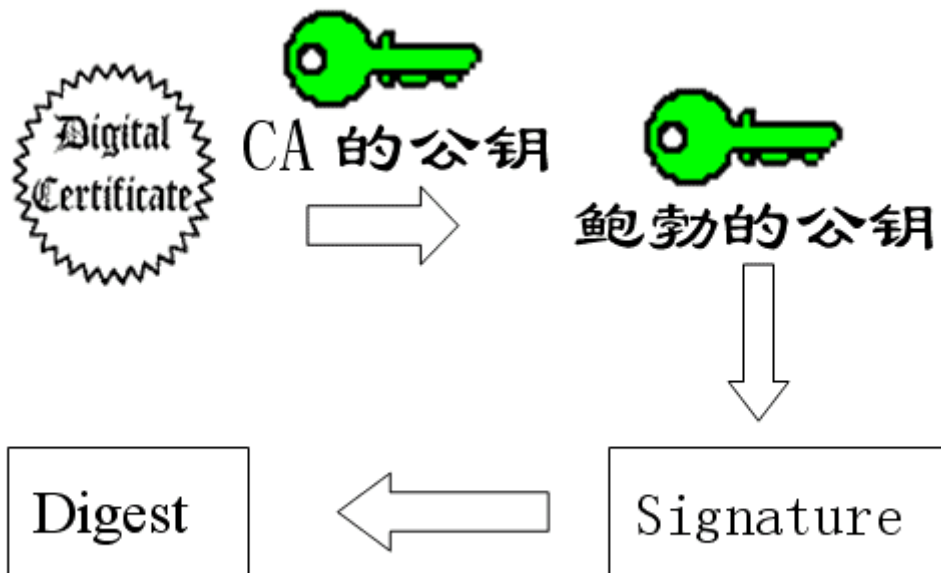


<http://blog.csdn.net/hp910315>

(2) 有数字证书



<http://blog.csdn.net/hp910315>



<http://blog.csdn.net/hp910315>

ssl协议

先来做一个约定：在网络通信过程中，发送数据的一方称为客户端，接收数据的一方称为服务端，从网络中窃取数据的一方称为第三方。（**传输加密与身份验证功能**）

SSL协议采用**对称加密来保护数据**，客户端和服务端在发送数据之前，协商出一个密钥，发送者用这个密钥对数据加密，之后再扔到网络中传输，服务端拿到数据后用相同的密钥解密，还原数据内容。

在**tcp三次握手时服务端发送数字证书**给客户端，客户端根据CA公钥拿出服务端公钥，并确认身份，用**服务端公钥对对称密钥加密发给服务端**

在网络传输过程中，如果数据被第三方截获，他**没有对称密钥，是解不出来数据内容**的。因此，数据内容可以得到保护，不会泄漏。

如果第三方截获密文数据以后，尝试替换或者修改密文数据，客户端或服务端拿到数据以后，用对称密钥解不出来正确内容，立刻就能知道数据出现了问题！

会话ID (Session id)

如果客户端第一次连接到服务器，那么这个字段就会保持为空。上图中该字段为空，说明这是第一次连接到服务器。

如果该字段不为空，说明以前是与服务器有连接的，在此期间，**服务器将使用Session ID映射对称密钥**，并将**Session ID存储在客户端浏览器中**，为映射设置一个时间限。如果**浏览器将来连接到同一台服务器（在时间到期之前）**，它将发送Session ID，服务器将对映射的Session ID进行验证，并使用以前用过的对称密钥来恢复Session，这种情况下不需要完全握手。也叫作**SSL会话恢复**



此时，就会出现两个问题：

解密数据以后，客户端和服务端怎么知道数据内容是否正确？

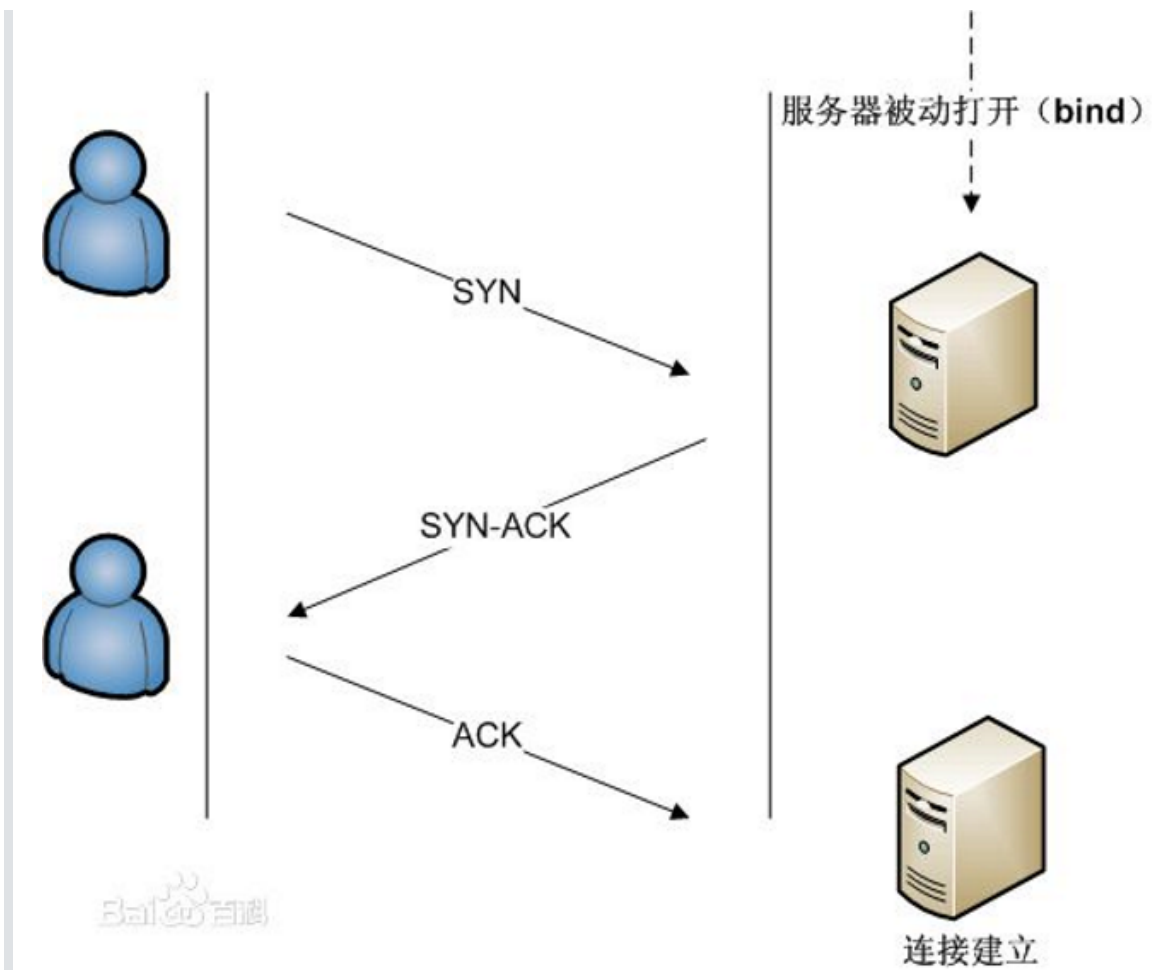
这个对称密钥是怎么协商出来的？

解析URL

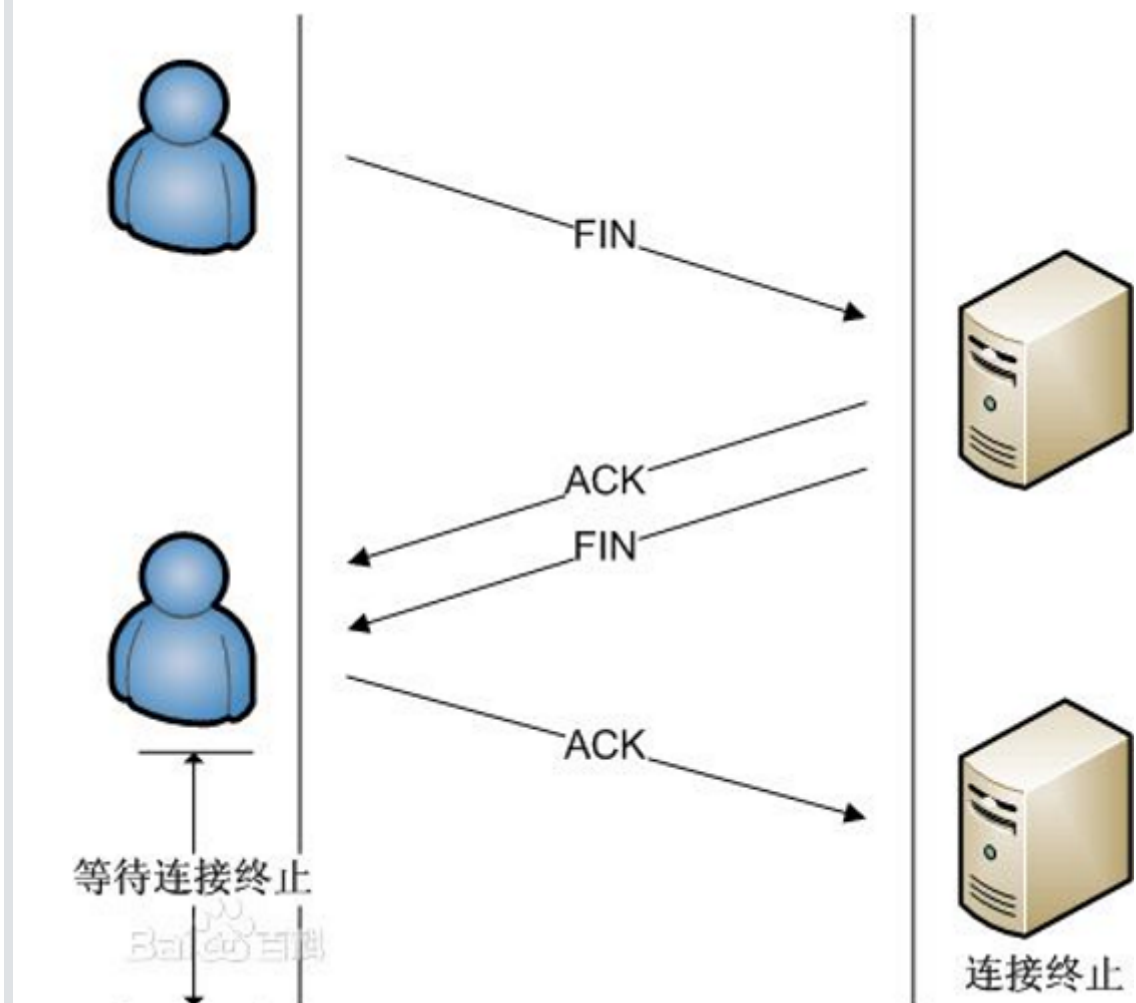
首先，浏览器向本地DNS服务器发送请求，如果本地没有缓存该域名的IP地址，就需要通过递归或迭代的方式向根域名服务器、顶级域名服务器、权威域名服务器发起查询请求，直至返回一个IP地址给浏览器。然后根据该IP地址以及端口号建立TCP连接，客户端发送HTTP请求，服务器处理请求并返回报文，再由浏览器进行解析渲染页面，最后TCP四次挥手结束连接

URL：协议、网络地址、资源路径

tcp连接：（为了向服务器发送请求）



关闭tcp连接:



服务器响应状态码：（200、404、500等）

	类别	原因短语
1xx	信息性状态码	接收的请求正在处理
2xx	成功状态码	请求正常处理完毕
3xx	重定向状态码	需要进行附加操作以完成请求
4xx	客户端错误状态码	服务器无法处理请求
5xx	服务器错误状态码	服务器处理请求出错

请求包含

请求报文包含哪几个部分（请求行、请求头、请求体、空行）

请求行：请求方法字段、URL字段和HTTP协议版本 例如：GET /index.html HTTP/1.1

响应报文(状态行、消息报头、响应正文)

内核缓存Recv-Q存储返回数据，然后程序从内核中进行IO读取数据（）

HTTP1.1与HTTP1.0

长连接：HTTP1.0默认使用短连接，每次请求都需要建立新的TCP连接，连接不能复用。HTTP1.1支持持久连接和请求的流水线处理，在一个TCP连接上可以传送多个HTTP请求和响应，减少建立和关闭TCP连接的消耗和延迟，提高效率。

host字段，新增一些错误通知状态码

带宽优化，缓存处理

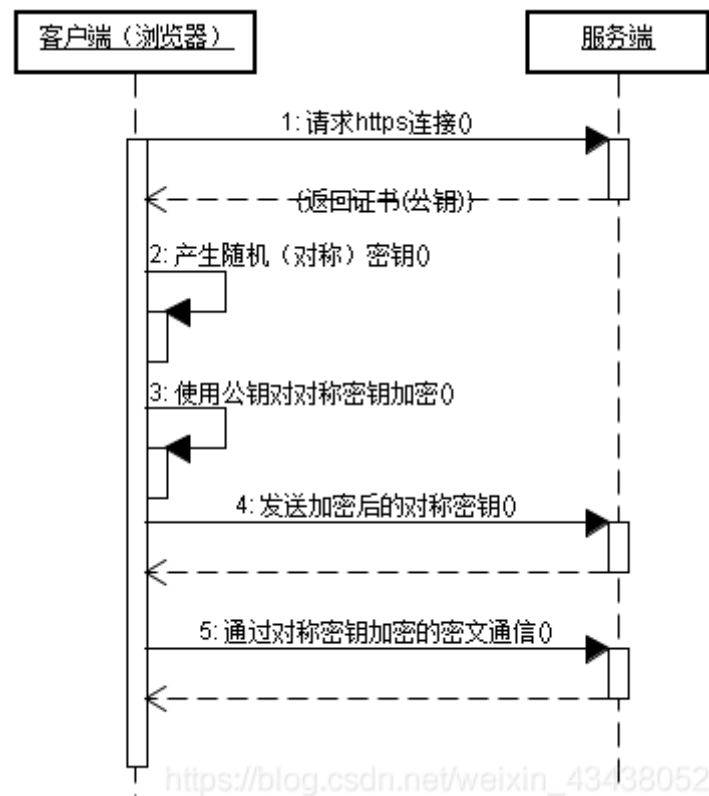
HTTP 与 HTTPS

- 1、HTTPS 协议需要到CA（Certificate Authority，证书颁发机构）申请证书，一般免费证书较少，因而需要一定费用。（以前的网易官网是http，而网易邮箱是 https。）
- 2、HTTP 是超文本传输协议，信息是明文传输，HTTPS 则是具有安全性的 SSL 加密传输协议。
- 3、HTTP 和 HTTPS 使用的是完全不同的连接方式，用的端口也不一样，前者是80，后者是443。
- 4、HTTP 的连接很简单，是无状态的。HTTPS 协议是由 SSL+HTTP 协议构建的可进行加密传输、身份认证的网络协议，比 HTTP 协议安全。（无状态的意思是其数据包的发送、传输和接收都是相互独立的。无连接的意思是指通信双方都不长久的维持对方的任何信息。）

HTTPS连接：

- 1、Client使用HTTPS的URL访问Web服务器，要求与Web服务器建立SSL连接。
- 2、Web服务器收到客户端请求后，会将网站的证书信息（证书中包含公钥）传送一份给客户端。

- 3、客户端的浏览器与Web服务器开始协商SSL连接的安全等级，也就是信息加密的等级。
- 4、客户端的浏览器根据双方同意的安全等级，建立会话密钥，然后利用网站的公钥将会话密钥加密，并传送给网站。
- 5、Web服务器利用自己的私钥解密出会话密钥。
- 6、Web服务器利用会话密钥加密与客户端之间的通信。
- 7、在交换密钥阶段使用公开密钥加密方式，之后建立通信交换报文阶段则使用共享密钥加密方式。

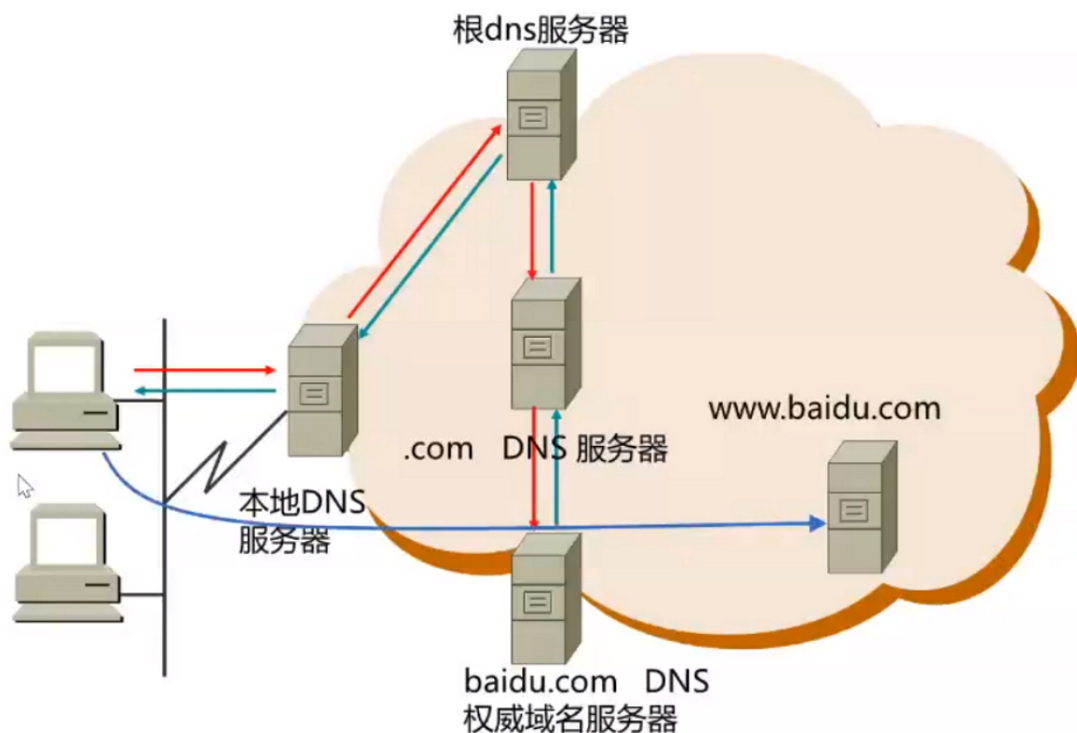


dns解析

1. 主机先向本地域名服务器进行递归查询
2. 本地域名服务器采用迭代查询，向一个根域名服务器进行查询
3. 根域名服务器告诉本地域名服务器，下一次应该查询的顶级域名服务器的IP地址
4. 本地域名服务器向顶级域名服务器进行查询
5. 顶级域名服务器告诉本地域名服务器，下一步查询权限服务器的IP地址
6. 本地域名服务器向权限服务器进行查询
7. 权限服务器告诉本地域名服务器所查询的主机的IP地址
8. 本地域名服务器最后把查询结果告诉主机

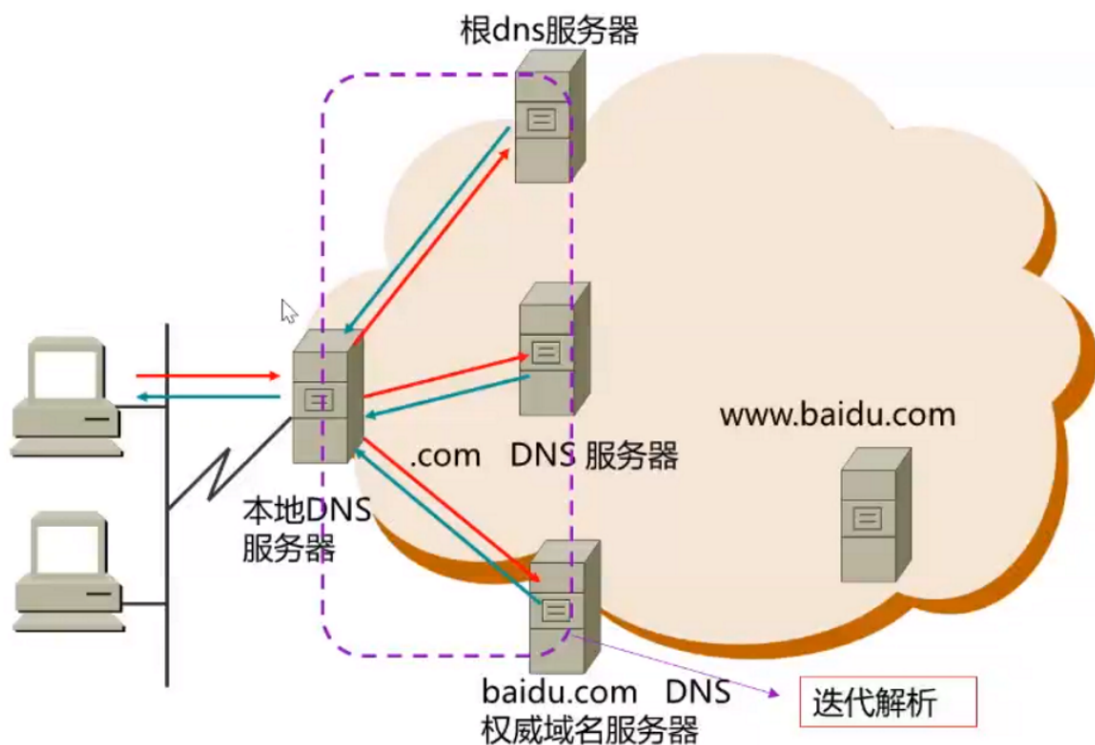
一、主机向本地域名服务器的查询一般都是采用递归查询。

所谓递归查询就是：如果主机所询问的本地域名服务器不知道被查询的域名的IP地址，那么本地域名服务器就以DNS客户的身份，向其它根域名服务器继续发出查询请求报文(即替主机继续查询)，而不是让主机自己进行下一步查询。因此，递归查询返回的查询结果或者是所要查询的IP地址，或者是报错，表示无法查询到所需的IP地址。



二、本地域名服务器向根域名服务器的查询的迭代查询。

迭代查询的特点：当根域名服务器收到本地域名服务器发出的迭代查询请求报文时，要么给出所要查询的IP地址，要么告诉本地服务器：“你下一步应当向哪一个域名服务器进行查询”。然后让本地服务器进行后续的查询。根域名服务器通常是把自己知道的顶级域名服务器的IP地址告诉本地域名服务器，让本地域名服务器再向顶级域名服务器查询。顶级域名服务器在收到本地域名服务器的查询请求后，要么给出所要查询的IP地址，要么告诉本地服务器下一步应当向哪一个权威域名服务器进行查询。最后，知道了所要解析的IP地址或报错，然后把这个结果返回给发起查询的主机。



GET和POST

1. get是获取数据的，而post是提交数据的。 2. GET 用于获取信息，是无副作用的，是幂等的，且可缓存，而POST 用于修改服务器上的数据，有副作用，非幂等，不可缓存。

Cookie和Session

回答： session 在服务器端， cookie 在客户端（浏览器）

session 默认被存储在服务器的一个文件里（不是内存）

session 的运行依赖 session id，而 session id 是存在 cookie 中的，也就是说，如果浏览器禁用了 cookie，同时 session 也会失效（但是可以通过其它方式实现，比如在 url 中传递 session_id）

session 可以放在 文件、数据库、或内存中都可以。

用户验证这种场合一般会用 session

(1)cookie数据存放在客户的浏览器上，session数据放在服务器上

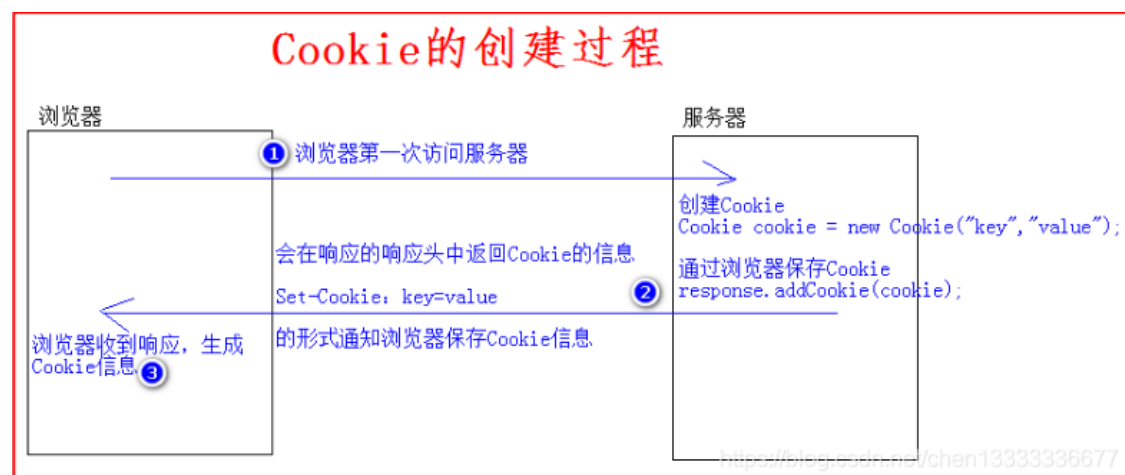
(2)cookie不是很安全，别人可以分析存放在本地的COOKIE并进行COOKIE欺骗,如果主要考虑到安全应当使用session

(3)session会在一定时间内保存在服务器上。当访问增多，会比较占用你服务器的性能，如果主要考虑到减轻服务器

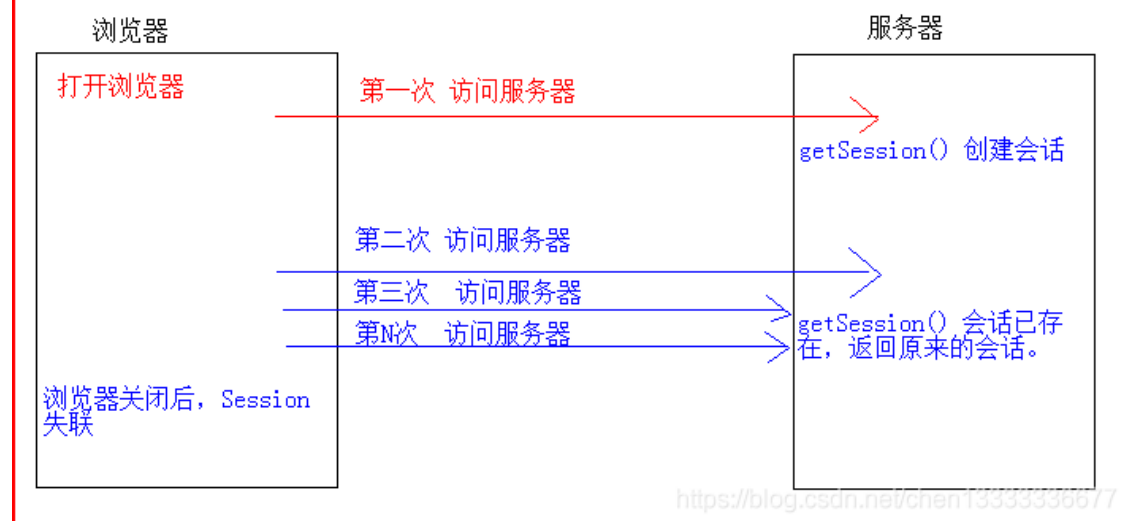
性能方面，应当使用COOKIE

(4)单个cookie在客户端的限制是3K，就是说一个站点在客户端存放的COOKIE不能3K。

(5)所以：将登陆信息等重要信息存放为SESSION;其他信息如果需要保留，可以放在COOKIE中



Session会话的创建和获取

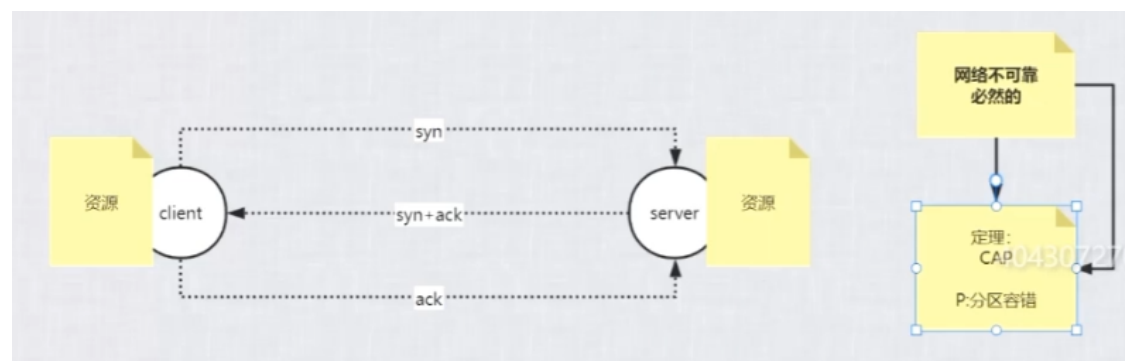


TCP与UDP区别

- 1、TCP **面向连接**, 通过 **三次握手** 建立连接, **四次挥手** 解除连接; UDP是 **无连接** 的, 即发送数据之前不需要建立连接, 这种方式为UDP带来了 **高效的传输效率**, 但也导致 **无法确保数据的发送成功**。
(无连接的意思是指通信双方都不长久的维持对方的任何信息)
- 2、TCP是 **可靠的** 通信方式。通过TCP连接传送的数据, TCP通过 **超时重传、数据校验、拥塞控制** 等方式来 **确保数据无差错, 不丢失, 不重复, 且按序到达**; 而UDP由于无需连接的原因, 将会以最大速度进行传输, 但不保证可靠交付, 也就是会出现 **丢失、重复** 等问题。
- 3、TCP面向 **字节流**, 实际上是TCP把数据看成一连串无结构的字节流, 由于连接的问题, 当网络出现波动时, 连接可能出现响应问题; UDP是面向 **报文的**, UDP没有拥塞控制, 因此网络出现拥塞不会使源主机的发送速率降低。
- 4、每一条TCP连接 **只能是点到点的**; 而UDP不建立连接, 所以可以 **支持一对一, 一对多, 多对一和多对多** 的交互通信, 也就是可以同时接受多个人的包。
- 5、TCP的逻辑通信 **信道是全双工的可靠信道**, UDP则是 **不可靠信道**。

三次握手

数据包的交换过程,



SYN攻击是什么? SYN 攻击是一种典型的 DoS/DDoS 攻击。

1. Client伪造大量的虚假ip,向Server发送SYN包

2. Server在接收到SYN包后,会返回响应,并进入SYN_RECV状态,等待客户端的确认
3. 但是伪造的ip肯定**不会给予响应**,于是Server以为数据包丢失,**不断重发,直到超时**

检测 SYN 攻击非常的方便, 当你在服务器上看到大量的半连接状态时, 特别是源IP地址是随机的, 基本上可以断定这是一次SYN攻击

防御方法:

- 缩短超时 (SYN Timeout) 时间
- 增加最大半连接数
- 过滤网关防护
- SYN cookies技术

为什么TIME_WAIT状态需要经过2MSL才能返回到CLOSE状态?

理论上, 四个报文都发送完毕, 就可以直接进入CLOSE状态了, 但是可能网络是不可靠的, 有可能最后一个ACK丢失。所以**TIME_WAIT状态就是用来重发可能丢失的ACK报文。一但这个ACK丢失的话, 服务器就无法正常的进入关闭连接状态。**

TCP 如何保证可靠性:

- 1、超时重传 (丢包)、数据校验、连续ARQ协议 (对于接收到的多个相同数据, 只确认最后一个, 根据最后一个回复确认)
- 2、流量控制: TCP支持根据接收端的处理能力, 来决定发送端的发送速度。tcp通过滑动窗口来进行流量控制
- 3、拥塞控制: 为了避免由于网络拥堵而采取的对发送方发送速率的限制称为拥塞控制。TCP 通过拥塞窗口实现拥塞避免。

TCP的四种拥塞控制算法 (不进行拥塞控制就会出现分组丢失 (丢包)、吞吐量为零 (死锁))

1. **慢开始** (拥塞窗口**指数增长**, >慢开始门限停止)
2. **拥塞避免** (拥塞窗口**线性增长**, 直到出现丢包)
3. **快重传** (发送方只要连续收到**3个重复确认**, **立即重传丢失数据**)
4. **快恢复** (出现拥塞不进入慢开始, **拥塞窗口减半后进入拥塞避免**)

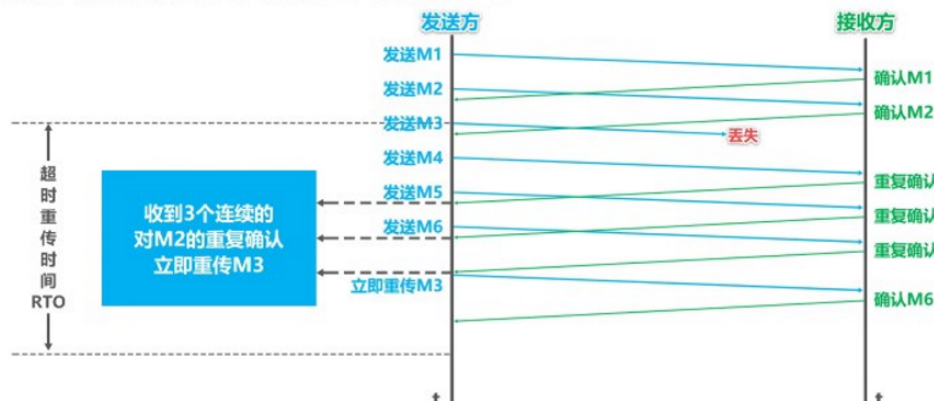
网络拥塞判断: 没有收到确认报文

拥塞窗口: 取决于网络拥塞情况而动态变化 (发送方)

滑动窗口: 传输层进行流量控制的一种措施, **接收方通过通告发送方自己的窗口大小, 从而控制发送方的发送速度**, 防止发送方发送速度过快而导致自己被淹没。

慢开始门限: 慢开始门限的初始值设置为16个报文段, 即 $cwnd = 16$, 网络出现超时 (这很可能就是网络发生拥塞了), 更新后的 $ssthresh$ 值为拥塞窗口的一半

快重传: 发送方只要连续收到3个重复确认, 立即重传丢失数据

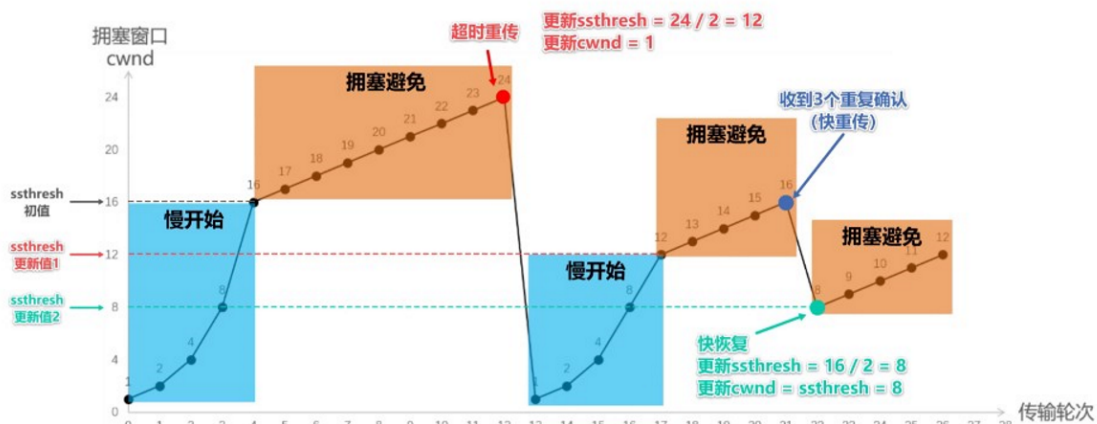


TCP的拥塞控制



- 发送方维护一个叫做**拥塞窗口cwnd**的状态变量，其值取决于网络的拥塞程度，并且动态变化。
 - 拥塞窗口**cwnd**的维护原则：只要网络**没有出现拥塞**，拥塞窗口就再增大一些；但只要网络**出现拥塞**，拥塞窗口就减少一些。
 - 判断出现**网络拥塞**的依据：没有按时收到应当到达的确认报文（即**发生重传**）。
- 发送方将拥塞窗口作为发送窗口，即 $swnd = cwnd$ 。
- 维护一个慢开始门限**ssthresh**状态变量：
 - 当 $cwnd < ssthresh$ 时，使用慢开始算法；
 - 当 $cwnd > ssthresh$ 时，停止使用慢开始算法而改用拥塞避免算法；
 - 当 $cwnd = ssthresh$ 时，既可使用慢开始算法，也可使用拥塞避免算法。

TCP的拥塞控制



TCP粘包

客户端发送两个独立的数据包，分别是 D1 和 D2

服务端一次接收到了两个数据包，D1 和 D2 粘合在一起，称之为 **TCP 粘包**

服务端分两次读取到了数据包，第一次读取到了完整的 D1 包和 D2 包的部分内容，第二次读取到了 D2 包的剩余内容，这称之为 **TCP 拆包**

TCP粘包现象处理方法

固定发送信息**长度**，或在两个信息之间加入**分隔符**

其他协议

地址解析协议ARP:

根据IP地址解析出物理地址, 在主机ARP高速缓存中存在IP地址映射物理地址的映射表, 若没有查询的相应的映射关系则执行ARP请求分组, 广播ARP请求, 向其他主机发送广播, 收到响应后在映射表中存储映射关系。

网际控制报文协议ICMP: 报告查错情况, 提供异常情况报告

——子网掩码的读法——

把子网掩码转换成二进制, 看到有多少个1斜杠/后面就填写多少数字

例如: (1) 192.168.1.0

255.255.255.0转换成二进制后为11111111.11111111.11111111.00000000里面共有24个1则可以写成 192.168.1.0/24

(2) 192.168.1.0/24

24代表子网掩码中前24位是1, 即11111111.11111111.11111111.00000000=255.255.255.0

(3) 可以和202.115.112.218/28直接通信的主机号是?

子网掩码11111111.11111111.11111111.11110000

则主机号前三个字节为202.115.112....

218=1101 1010 与11110000相比得出1101 0000(208)<1101 <1110 0000(224)

所以可以和202.115.112.218/28直接通信的主机号是202.115.112.208/28~202.115.112.224/28

- IP地址: 类似于你这台电脑的标志, 但在网络上是靠IP地址识别的。如果利用TCP/IP协议组网, 那么一个网段内的所有电脑都必须有一个IP地址, 并且不能重复。
- 子网掩码和IP地址是配合一起的, 将IP地址分成两段, 网络段和主机段。
- 例如你的IP地址是192.168.1.2, 子网掩码是255.255.255.0, 那么子网掩码全是255的对应的IP地址段表示网络段, 是0的对应的IP地址段表示主机段, 以上为例, 则192.168.1表示网络, 2表示主机。
- 如果需要在网络内新增一台主机, 则只要改变它也只能改变最后一位。这样才能保证在同一网络。

B类地址中, 后16位为主机地址, 255.255.0.0, 二进制为11111111 11111111 00000000 00000000

要想切割成10个子网, 至少要向主机位借4位, $2^4 = 16 > 10$

则子网掩码设置成20位, 即二进制为 11111111 11111111 11110000 00000000, 再换算成十进制之后为: 255.255.240.0

mysql

数据库分类

关系型数据库: SQL (Structured Query Language)

MySQL、Oracle、Sql Server、DB2、SQLite

通过表和表之间，行和列之间的关系进行数据的存储

通过外键关联来建立表与表之间的关系

非关系型数据库：NoSQL (Not Only SQL)

Redis、MongoDB

指数据以对象的形式存储在数据库中，而对象之间的关系通过每个对象自身的属性来决定

数据库存储引擎

INNODB

支持事务安全、支持主外键，支持事务的处理

不仅缓存索引,还要缓存真实数据,对内存要求比较高,而且内存大小对性能有决定性的影响

行锁 操作时,只锁某一行,不对其它行有影响 适合高并发的操作

MYISAM

节约空间，速度较快，**表锁** 操作一条记录也会锁住整个表 不适合高并发的操作

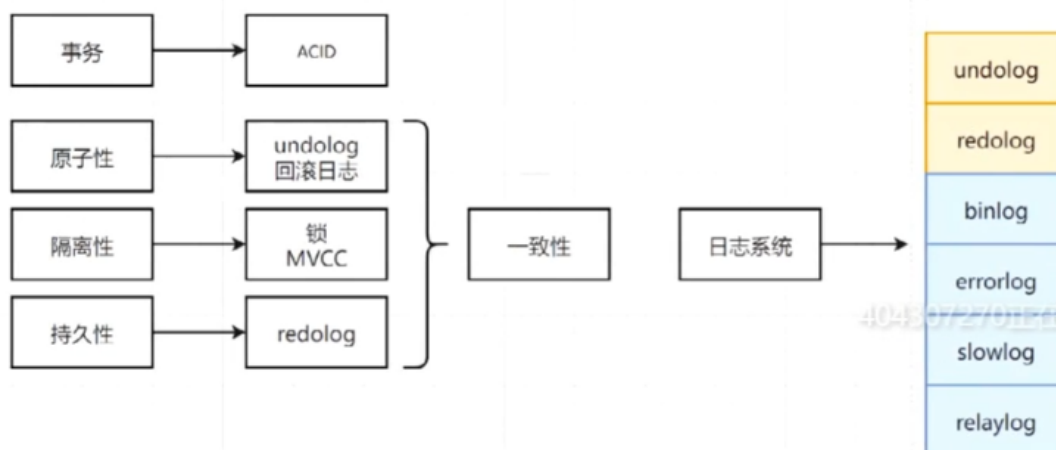
InnoDB与MyISAM的最大不同点:一是支持事务,二是采用了行级锁。

sql的执行要经过**连接器**、**分析器**（解析成解析树，词法语法解析判断，预处理器再进行判断）、**优化器**（多条查询路径选择最优的）、**执行器**（查buffer缓存，默认关闭）

事务

要么都成功，要么都失败（例如银行转账：只有A转账成功且B成功到账，该事件才算结束，如果一方不成功，则该事务不成功 //单条执行破坏原子性，不符合事务）

```
20
21 ##MySQL如何手动开启事务? start TRANSACTION;
22 begin;
23 update cmb_user set money = money - 200 where userid=1;
24 update cmb_user set money = money + 200 where userid=2;
25 commit;
26 rollback;
```



数据库数据修改是随机读写，写日志时是顺序读写（append追加）

随机读写：不断的写入、删除数据，就会产生很多零零散散的存储空间，就会造成一个较大的数据文件放在许多不连续的存贮空间上，读写这部分数据时，就是随机读写，磁头要不断的调整磁道的位置，以在不同位置上的读写数据，相对于连续空间上的顺序读写，要耗时很多

顺序读写：与随机读写不同的是，顺序读写是优先分配一块文件空间，然后后续内容追加到对应空间内。在使用顺序IO进行文件读写时候，需要知道上次写入的地方，所以需要维护一个索引或者轮询获得一个没有写入位置。

一、事务的基本要素（ACID）

- 1、原子性（Atomicity）：事务开始后所有操作，要么全部做完，要么全部不做，不可能停滞在中间环节。事务执行过程中出错，会回滚到事务开始前的状态，所有的操作就像没有发生一样。也就是说事务是一个不可分割的整体，就像化学中学过的原子，是物质构成的基本单位。
- 2、一致性（Consistency）：事务开始前和结束后，数据库的完整性约束没有被破坏。比如在一个事务中A向B转账，不可能A扣了钱，B却没收到。
- 3、隔离性（Isolation）：同一时间，只允许一个事务请求同一数据，不同的事务之间彼此没有任何干扰。比如A正在从一张银行卡中取钱，在A取钱的过程结束前，B不能向这张卡转账。
- 4、持久性（Durability）：事务完成后，事务对数据库的所有更新将被保存到数据库，不能回滚。

二、事务的并发问题

- 1、脏读：事务A读取了事务B更新的数据，然后B回滚操作，那么A读取到的数据是脏数据
- 2、不可重复读：事务A多次读取同一数据，事务B在事务A多次读取的过程中，对数据作了更新并提交，导致事务A多次读取同一数据时，结果不一致。
- 3、幻读：系统管理员A将数据库中所有学生的成绩从具体分数改为ABCDE等级，但是系统管理员B就在这个时候插入了一条具体分数的记录，当系统管理员A改结束后发现还有一条记录没有改过来，就好像发生了幻觉一样，这就叫幻读。（快照读与当前读一起使用就会产生幻读）

小结：不可重复读的和幻读很容易混淆，不可重复读侧重于修改，幻读侧重于新增或删除。解决不可重复读的问题只需锁住满足条件的行，解决幻读需要锁表

三、事务的隔离级别

三、MySQL事务隔离级别

事务隔离级别	脏读	不可重复读	幻读
读未提交（read-uncommitted）	是	是	是
不可重复读（read-committed）	否	是	是
可重复读（repeatable-read）	否	否	是
串行化（serializable）	否	否	否

读未提交：读到未提交事务数据

串行化：底层对每条语句加锁（读锁或写锁）

更改事务级别：数据库直接更改配置参数、@Transactional(isolation=)在项目中加spring注解更改数据库级别

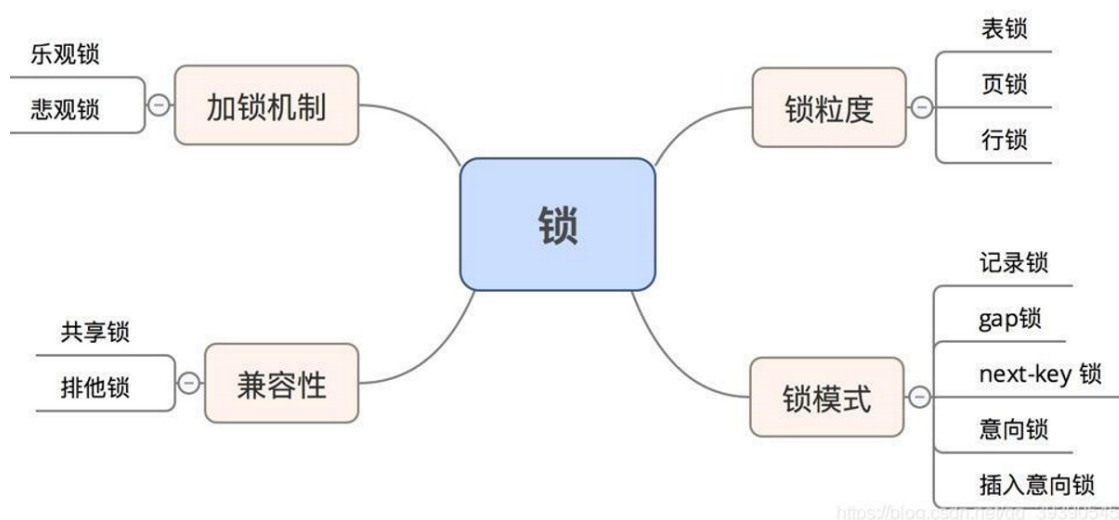
(实质是使用了 JDBC 的事务来进行事务控制的，基于 Spring 的动态代理的机制)

mysql默认隔离级别是repeatable read，因为serializable的缺点很明显，虽然能大大提高安全性，但效率太低了。

如果追求性能隔离级别选择RC，如果是报表统计，需要统计某一时刻的统一数据，选择RR开启事务读快照

注意：serializable名字叫可串行化，并不是说使用了serializable隔离级别所有的事务都要串行执行，它允许事务并发执行，但系统必须保证并发调度是可串行的，即我允许你并行执行，但要求你的并发调度不致发生错误，即如果你的另一个事务只进行查数据，那当然对我的事务无影响，我就允许你执行，但如果你另一个事务有增删改这些操作，就会被阻塞以保障不发生错误。

四、mysql锁



锁的粒度：表锁 > 行锁 -- 这是因为表锁会锁定更多的记录以及资源因此粒度比较大

加锁效率：表锁 > 行锁 -- 这是因为表锁直接锁定了整个表资源而不需要向行锁一样一行行锁

冲突概论：表锁 > 行锁 -- 锁整张表数据所有写的操作都需要阻塞因此冲突更多

并发性能：表锁 < 行锁 -- 行锁的冲突概率小自然并发高

(MyISAM只支持表锁)

共享锁 (S)： 同时访问数据但是不可修改数据

排他锁 (X)： 如果一个事物获取了一行数据的排他锁，其他事物不可以读或者写

意向锁 (IX、IS)： 判断表中是否有锁

悲观锁 (Pessimistic Lock)：就是很悲观，每次去拿数据的时候都认为别人会修改。所以每次在拿数据的时候都会上锁。这样别人想拿数据就被挡住，直到悲观锁被释放，悲观锁中的共享资源每次只给一个线程使用，其它线程阻塞，用完后再把资源转让给其它线程，但是在效率方面，处理加锁的机制会产生额外的开销，还有增加产生死锁的机会。另外还会降低并行性，如果已经锁定了一个线程A，其他线程就必须等待该线程A处理完才可以处理

例如：行锁，表锁，读锁（共享锁），写锁（排他锁），以及synchronized

乐观锁 (Optimistic Lock)：就是很乐观，每次去拿数据的时候都认为别人不会修改。所以不会上锁，但是如果想要更新数据，则会在更新前检查在读取至更新这段时间别人有没有修改过这个数据。如果修改过，则重新读取，再次尝试更新，循环上述步骤直到更新成功（当然CAS：Compare-and-Swap，即比较并替换，也有叫做Compare-and-Set的，比较并设置。实现乐观锁原理过程。

- 1、比较：读取到了一个值A，在将其更新为B之前，检查原值是否仍为A（未被其他线程改动）。
- 2、设置：如果是，将A更新为B，结束。[1]如果不是，则什么都不做。

- 1、乐观锁并未真正加锁，所以效率高。一旦锁的粒度掌握不好，更新失败的概率就会比较高，容易发生业务失败。
- 2、悲观锁依赖数据库锁，效率低。更新失败的概率比较低。

死锁

死锁发生的条件主要有三种：1)互斥 2)不可剥夺 3)请求和保持 4)循环等待

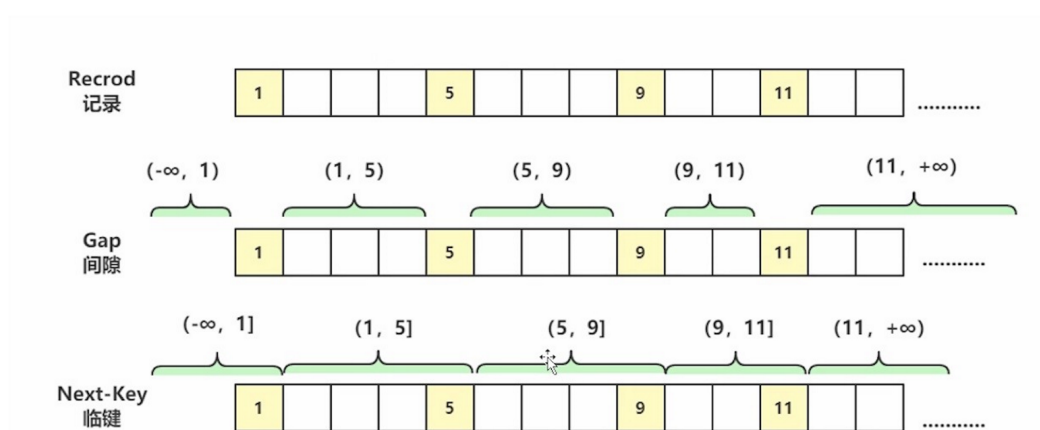
锁类型	读锁	写锁	意向读锁	意向写锁
读锁	兼容	冲突	兼容	冲突
写锁	冲突	冲突	冲突	冲突
意向读锁	兼容	冲突	兼容	兼容
意向写锁	冲突	冲突	兼容	兼容

意向锁之间不冲突，意向写锁与读写锁冲突，写锁与其他锁冲突

如何结束阻塞：Commit或者Rollback

锁区间

锁的区间



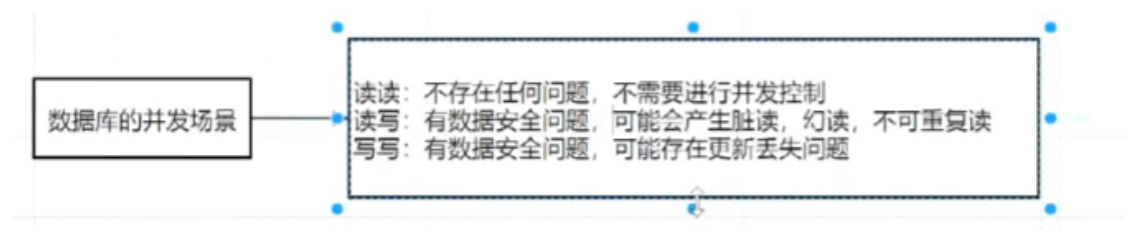
记录锁：

间隙锁：只存在RR隔离级别，（实际存在的间隙区间）

临键锁：（实际存在的临键区间）

MVCC

数据库并发问题



MVCC 主要用于解决并发读写问题，维持了一个数据的多个版本，使得读写操作间没有冲突，主要由隐藏字段（事务ID，回滚指针，隐藏主键），undolog，readview 三部分组成

COW，读的是历史版本数据，写的是新的数据

快照读：普通select查询（读取历史版本数据）（nacos服务注册表COW类似，读的是历史数据，写的是副本数据，然后替换，容忍我们读到旧数据，都是为了解决高并发下的数据读写问题）

当前读：insert、update等操作前需要读取数据的方式（悲观锁）（读取的都是最新的数据，不会读到历史数据）

（快照生成时间是begin后执行第一个快照读语句是生成）

Undo.log：数据+trx_id(事务id)+roll_pointer(回滚指针)、每条记录以版本链即roll_pointer连接（对于同一个数据的不同版本组成的链表，链首是最新的旧记录，链尾是最旧的旧记录）（为了避免占用内存，有一个线程purge定时清理无用数据（事务提交，不需要回滚的数据））

快照（readview）：（通过可见性算法决定读取到的数据）

m_ids（在进行读操作时还存在的事务ID列表）

min_trx_id（m_ids中最小的）

max_trx_id

creator_trx_id(查询操作所在事务id) 与undo.log中的trx_id进行比较

RC：快照以select为单位（语句级别快照）

RR：快照以事务为单位（事务级别快照），因为一个事务里的所有查询是使用同一个readview，所以解决了不可重复读问题以及幻读（间隙锁）

RC:每次都能读取到已经提交的数据，会产生不可重复读问题，每次进行快照读都会生成新的readview

RR:解决了不可重复读的问题，同一个事务中多次查询读取的结果是一样，原因是读取的历史记录，只有在第一次进行快照读的时候会生成readview，之后的快照读都会沿用第一次生成的readview（幻读问题通过加锁解决，阻塞其他修改事务）

生成快照的时机不同

五、ACID保证

一、Mysql怎么保证一致性的？

一致性的话应该分两个层面来保证。

1.从数据库层面，数据库通过原子性、隔离性、持久性来保证一致性。也就是说ACID四大特性之中，C(一致性)是目的，A(原子性)、I(隔离性)、D(持久性)是手段，是为了保证一致性，数据库提供的手段。**数据库必须要实现AID三大特性，才有可能实现一致性**。例如，原子性无法保证，显然一致性也无法保证。

2.从应用层面，通过代码判断数据库数据是否有效，然后决定回滚还是提交数据（一个事务两动作）！

二、Mysql怎么保证原子性的？

是利用Innodb的undo log。

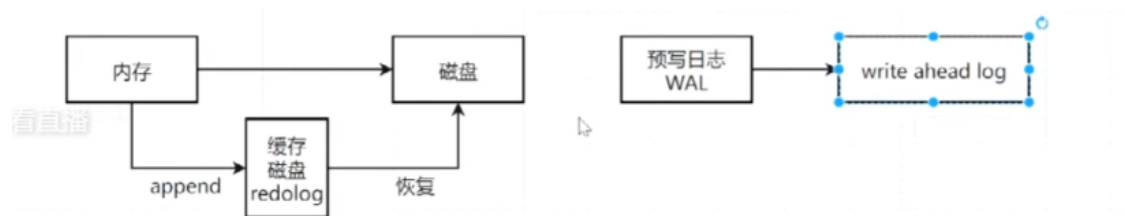
undo log名为回滚日志，是实现原子性的关键，（**回滚指针相当于delect**）undo log记录了这些回滚需要的信息，当事务执行失败或调用了rollback，导致事务需要回滚，便可以利用**undo log中的信息将数据回滚到修改之前的样子**。

三、Mysql怎么保证持久性的？

是利用Innodb的**redo log（类似于磁盘缓存）**。

正如之前说的，Mysql是先把磁盘上的数据加载到内存中，在内存中对数据进行修改，再刷回磁盘上。如果此时突然宕机，内存中的数据就会丢失。

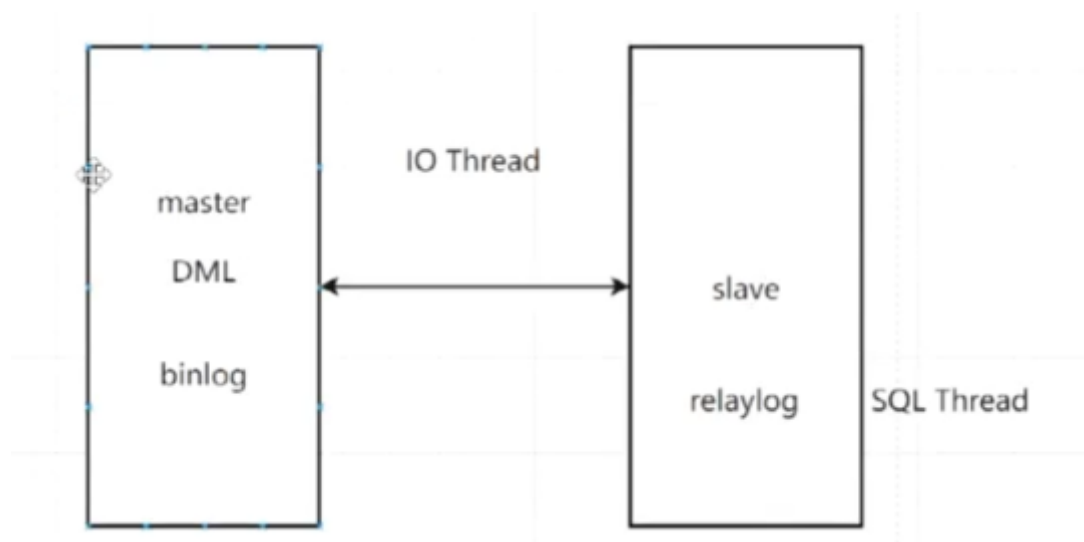
当数据修改的时候，不仅在内存中操作，还会在redo log中记录这次操作。当事务提交的时候，会将redo log日志进行刷盘(redo log一部分在内存中，一部分在磁盘上)。当数据库宕机重启的时候，**会将redo log中的内容恢复到数据库中，再根据undo log和binlog内容决定回滚数据还是提交数据**。



什么是binlog **(主从之间)**

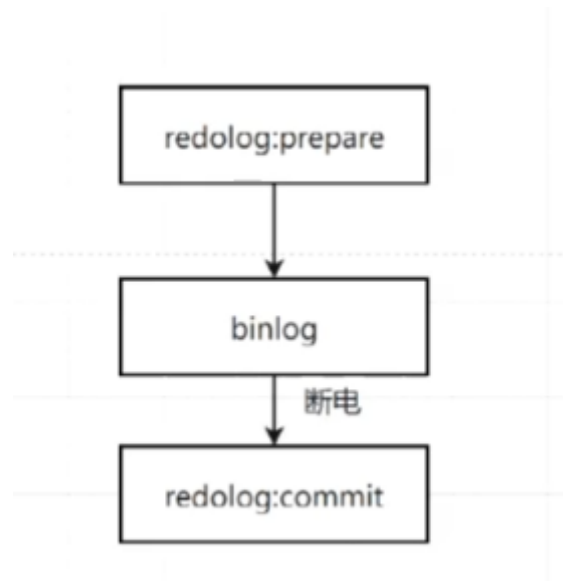
mysql中有各种类型的日志，**记录了mysql包括启动、运行、连接、更改等各种操作**，而binlog就是其中最重要的一种日志，它**记录了MySQL所有数据的变更，并以二进制的形式存储在磁盘上**

binlog 的主要目的是**复制和恢复**。



两阶段提交：

如果prepare中有数据，而binlog没有则丢弃



长事务缺点（超过八秒）：

并发情况下，数据库连接池容易被撑爆

锁定太多的数据，造成大量的阻塞和锁超时

执行时间长，容易造成主从延迟

回滚所需要的时间比较长

undo log膨胀

容易导致死锁

优化：

长事务分解，非核心业务以及非数据库操作放事务外边

如果事务中需要锁多个行，要把最可能造成锁冲突、最可能影响并发度的锁尽量往后放。（开日志，买票付款，卖家收款（有锁冲突，放最后））

索引

mysql索引结构

mysql的索引就是一种树状结构 （核心理念：空间换时间）

mysql的索引没有采用平衡二叉查找树。

索引是硬盘中的数据，每次查找树的一个节点就要做一次io操作，一次磁盘操作的成本非常的大。

mysql为了减少io操作把索引没有建成平衡二叉查找树。

而是选用了b+树，b+树是有b树转换过来的。所以我们先了解了b树。

b树和平衡二叉查找树的区别。b树的一个节点上面可以存多个数据。b树比平衡二叉查找树更加的矮和胖。

B+树是基于b树的一种数据结构，他的每个非叶子节点上面不存储数据，只存储索引，具体的数据都在叶子节点上面，b+树的兄弟节点之间是用双向指针链接起来的，这样方便范围查找

一次加载数据单位: **Data page 16K (mysql)** (内存与磁盘 page 4k) 最大qps 10万
redis qps 4-10万

MyISAM引擎使用B+Tree作为索引结构, 叶节点的data域存放的是数据记录的地址。

MyISAM索引文件和数据文件是分离的, 索引文件仅保存**数据记录的地址**。

MyISAM的索引方式也叫做“非聚集”的。

InnoDB的数据文件本身就是索引文件, 这种索引叫做聚集索引。

1、索引的优势

- (1) 提高查询效率 (降低IO使用率)
- (2) 降低CPU使用率

比如查询order by age desc, 因为B+索引树本身就是排好序的, 所以再查询如果触发索引, 就不用再重新查询了。

2、索引的弊端

- (1) 索引本身很大, 可以存放在内存或硬盘上, 通常存储在硬盘上。
- (2) 索引不是所有情况都使用, 比如①少量数据②频繁变化的字段③很少使用的字段
- (3) 索引会降低增删改的效率

3、索引的分类

- (1) 单值索引
- (2) 唯一索引
- (3) 联合索引: 两个或两个以上字段联合组成一个索引
- (4) 主键索引

普通索引: 普通索引是 MySQL 中最基本的索引类型, 它没有任何限制, 唯一任务就是加快系统对数据的访问速度。普通索引允许在定义索引的列中插入重复值和空值。

唯一索引: 唯一索引与普通索引类似, 不同的是创建唯一性索引的目的不是为了提高访问速度, 而是为了避免数据出现重复。唯一索引列的值必须唯一, 允许有空值。如果是组合索引, 则列值的组合必须唯一。创建唯一索引通常使用 UNIQUE 关键字。例如在student 表中的 id 字段上建立名为 index_id 的索引|CREATE UNIQUE INDEX index_id ON tb_student(id);

主键索引: 主键索引就是专门为主键字段创建的索引, 也属于索引的一种。主键索引是一种特殊的唯一索引, 不允许值重复或者值为空。创建主键索引通常使用 PRIMARY KEY 关键字。不能使用 CREATE INDEX 语句创建主键索引。

空间索引: 空间索引是对空间数据类型的字段建立的索引, 空间索引主要用于地理空间数据类型, 很少用到。

全文索引: 全文索引主要用来查找文本中的关键字, 只能在 CHAR、VARCHAR 或 TEXT 类型的列上创建。在 MySQL 中只有 MyISAM 存储引擎支持全文索引。全文索引允许在索引列中插入重复值和空值。

备注: 唯一索引和主键索引唯一的区别: 主键索引不能为null

自增主键和uuid作为主键的区别么? 由于主键使用了聚集索引, 如果主键是自增id, 那么对应的数据一定也是相邻地存放在磁盘上的, 写入性能比较高。如果是uuid的形式, 频繁的插入会使innodb频繁地移动磁盘块, 写入性能就比较低了。

索引下推

```
key 'Index_Uni' ('name','age')

select * from tuser where name like '张%' and age=10;
```

此时只有name的索引会生效，MySQL5.6之后采用了优化策略，在定位到'张%'后，无需立即回表查询对应age=10的记录，而是可以**先在索引树上遍历age=10**的记录再回表查询。

覆盖索引（查询联合索引中的部分或全部字段）

- 1.覆盖索引是一种**数据查询方式**，不是索引类型
- 2.在索引数据结构中，通过索引值可以直接找到要查询字段的值，而**不需要通过主键值回表查询**，那么就叫覆盖索引
- 3.查询的字段被使用到的索引树**全部覆盖到**

面试题

我们的主键为什么最好是趋势递增的？

防止索引树分裂与合并 提升我操作性能

一个表为什么达到500W 就最好分表？ 3层索引树到底能够支撑多少数据？

page页 数据是多大 目录记录是多大

假如page 16K 目录记录 16Byte 一行完整的数据 1K 3层索引树到底能够支撑多少数据？

16k /16B 1024 根目录 放1024个目录

第二层 1024 * 1024 个记录

第三层 1024 * 1024 * 16

聚集索引：存有真实的数据 没有主键

主键可以没有 聚集索引是必有的 rowid

二级索引 除了聚集索引以为都是二级索引 非聚集索引

二级索引树 我们会记录二级索引的所有字段 并且会记录聚集索引的值

建立了age name的联合索引

```
select * from teacher where age =18
```

age name address

查询的字段不在索引树里面，我需要去聚集索引查 回表

```
select age ,name from teacher where age =18
```

 减少回表

```
select age ,address from teacher where age =18
```

```
select age ,name from teacher where age =18
```

 减少回表

覆盖索引 不是索引 减少回表的方式去提升性能

```
select age ,address from teacher where age =18
```

优化 先查出ID 再跟ID具体查数据 (redis缓存) 分片思想

缺点：

- 创建和维护索引组要耗费时间，并且随着数据量的增加所耗费的时间也会增加。
- 索引需要占磁盘空间，除了数据表占数据空间以外，每一个索引还要占一定的物理空间。
- 当对表中的数据进行增加、删除和修改的时候，索引也要动态维护，这样就降低了数据的维护速度。

假如表里面有abc三个字段的联合索引 where c=1 and b>1 and a=1 能不能走到索引?

一般情况下, 会走到索引 但是不是全部字段都能走到索引

1.最左匹配原则 匹配的是索引字段的顺序

a b c

a能走到索引 b有, b也能走到 c有 c

2.范围查询会使下一个列索引列失效

where c=1 and b=1 and a=1 a b c

where c=1 and b=1 不能

where c=1 and a=1 a 下推 一种区别于索引的方式

like % huihui 会使当前索引列失效

3.类型转换 I

4.查询的数据超过30% (估值)

5.走不走 优化器说了算

假如, 你的sql很慢会怎么解决?

1.语句优化

a.尽量索引 索引是否失效 是否建立了正确的索引

b.减少回表 尽量走到覆盖索引

尽量去线上预执行

2.分片思想 单条很慢 分多次

分页

3.数据结构 数据冗余 (排行榜) 分库分表 (怎么做负载 关联数据怎么查询)

4.bufferpool 调大内存区间 redis 所有数据都是放在 内存

5.引入缓存组件 (redis、es)

索引失效场景

创建了索引并不意味着高枕无忧, 在很多场景下, 索引会失效。下面列举了一些导致索引失效的情形, 是我们写SQL语句时应尽量避免的。

1、条件字段原因

单字段有索引，WHERE条件使用多字段（含带索引的字段），例如 `SELECT * FROM student WHERE name = '张三' AND addr = '北京市'` 语句，如果name有索引而addr没索引，那么SQL语句不会使用索引。

多字段索引，违反最佳左前缀原则。例如，student表如果建立了(name,addr,age)这样的索引，WHERE后的第一个查询条件一定要是name，索引才会生效。

2、<>、NOT、in、not exists

当查询条件为等值或范围查询时，索引可以根据查询条件去找对应的条目。否则，索引定位困难（结合我们查字典的例子去理解），执行计划此时可能更倾向于全表扫描，这类的查询条件有：

<>、NOT、in、not exists

3、查询条件中使用OR

如果条件中有or，即使其中有条件带索引也不会使用(因此SQL语句中要尽量避免使用OR)。要想使用OR，又想让索引生效，只能将OR条件中的每个列都加上索引。

4、查询条件使用LIKE通配符

SQL语句中，使用后置通配符会走索引，例如查询姓张的学生 (`SELECT * FROM student WHERE name LIKE '张%'`)，而前置通配符(`SELECT * FROM student WHERE name LIKE '%东'`)会导致索引失效而进行全表扫描。

5、索引列上做操作（计算，函数，（自动或者手动）类型装换）

有以下几种例子：

在索引列上使用函数：例如 `select * from student where upper(name)='ZHANGFEI'`；会导致索引失效，而 `select * from student where name=upper('ZHANGFEI')`；是会使用索引的。

在索引列上计算：例如 `select * from student where age-1=17`；

6、在索引列上使用mysql的内置函数，索引失效

例如，`SELECT * FROM student WHERE create_time`

7、索引列数据类型不匹配

例如，如果age字段有索引且类型为字符串（一般不会这么定义，此处只是举例）但条件值为非字符串，索引失效，例如 `SELECT * FROM student WHERE age=18` 会导致索引失效。

8、索引列使用IS NOT NULL或者IS NULL可能会导致无法使用索引

B-tree索引IS NULL不会使用索引，IS NOT NULL会使用，位图索引IS NULL、IS NOT NULL都会使用索引。

最后，对索引的使用做一个总结吧：

索引有利于查询，但不能随意加索引，因为索引不仅会占空间，而且需要在写库时进行维护。

如果多个字段常常需要一起查询，那么在这几个字段上建立联合索引是个好办法，同时注意最左匹配原则。

不要在重复度很高的字段上加索引，例如性别。

避免查询语句导致索引失效

范式

第一范式（1NF）：要求数据库表的每一列都是不可分割的原子数据项。

举例说明：

学号	姓名	性别	家庭信息	学校信息
20150001	李白	男	3口人，北京	硕士，研二
20150002	杜甫	男	2口人，南京	本科，大四
20150003	王维	男	4口人，上海	本科，大三
20150004	白居易	男	3口人，河南	硕士，研一
20150005	刘禹锡	男	4口人，上海	本科，大一
20150006	李清照	女	5口人，西安	硕士，研三
20150007	苏轼	男	2口人，南京	大专，大二
20150008	屈原	男	4口人，吉林	博士，博五
20150009	陶渊明	男	1口人，长沙	博士，博三

在上面的表中，“家庭信息”和“学校信息”列均不满足原子性的要求，故不满足第一范式，调整如下：

学号	姓名	性别	家庭人口	户籍	学历	所在年级
20150001	李白	男	3口人	北京	硕士	研二
20150002	杜甫	男	2口人	南京	本科	大四
20150003	王维	男	4口人	上海	本科	大三
20150004	白居易	男	3口人	河南	硕士	研一
20150005	刘禹锡	男	4口人	上海	本科	大一
20150006	李清照	女	5口人	西安	硕士	研三
20150007	苏轼	男	2口人	南京	大专	大二
20150008	屈原	男	4口人	吉林	博士	博五
20150009	陶渊明	男	1口人	长沙	博士	博三

可见，调整后的每一列都是不可再分的，因此满足第一范式（1NF）；

第二范式（2NF）：在1NF的基础上，非码属性必须完全依赖于候选码（在1NF基础上消除非主属性对主码的**部分函数依赖**）

第二范式需要确保数据库表中的每一列都和主键相关，而不能只与主键的某一部分相关（主要针对联合主键而言）。

举例说明：

订单号	产品号	产品数量	产品折扣	产品价格	订单金额	订单时间
2008003	205	100	0.9	8.9	2870	20080103
2008003	206	200	0.8	9.9	2870	20080103
2008005	207	200	0.75	10	2000	20080203
2008006	207	400	0.85	12	4800	20080206
2008007	207	1000	0.88	14	14000	20080209
2008008	210	240	0.95	8	12255	20100423
2008008	211	300	0.75	8	12255	20100423
2008008	212	350	0.8	15.9	12255	20100423

在上图所示的情况中，同一个订单中可能包含不同的产品，因此主键必须是“订单号”和“产品号”联合组成，

但可以发现，产品数量、产品折扣、产品价格与“订单号”和“产品号”都相关，但是订单金额和订单时间仅与“订单号”相关，与“产品号”无关，

这样就不满足第二范式的要求，调整如下，需分成两个表：

订单号	产品号	产品数量	产品折扣	产品价格
2008003	205	100	0.9	8.9
2008003	206	200	0.8	9.9
2008005	207	200	0.75	10
2008006	207	400	0.85	12
2008007	207	1000	0.88	14
2008008	210	240	0.95	8
2008008	211	300	0.75	8
2008008	212	350	0.8	15.9

订单号	订单金额	订单时间
2008003	2870	20080103
2008003	2870	20080103
2008005	2000	20080203
2008006	4800	20080206
2008007	14000	20080209
2008008	12255	20100423
2008008	12255	20100423
2008008	12255	20100423

第三范式（3NF）：在2NF基础上，任何非主属性不依赖于其它非主属性（在2NF基础上消除**传递依赖**）

第三范式需要确保数据表中的每一列数据都和主键直接相关，而不能间接相关。

举例说明:

学号	姓名	性别	家庭人口	班主任姓名	班主任性别	班主任年龄
20150001	李白	男	3口人	陈洁	女	35
20150002	杜甫	男	2口人	陈洁	女	35
20150003	王维	男	4口人	陈洁	女	35
20150004	白居易	男	3口人	李丽	女	32
20150005	刘禹锡	男	4口人	李丽	女	32
20150006	李清照	女	5口人	王安	男	29
20150007	苏轼	男	2口人	南林	男	34
20150008	屈原	男	4口人	南林	男	34
20150009	陶渊明	男	1口人	王安	男	29

上表中,所有属性都完全依赖于学号,所以满足第二范式,但是“班主任性别”和“班主任年龄”直接依赖的是“班主任姓名”,而不是主键“学号”,所以需做如下调整:

学号	姓名	性别	家庭人口	班主任姓名	班主任姓名	班主任性别	班主任年龄
20150001	李白	男	3口人	陈洁	陈洁	女	35
20150002	杜甫	男	2口人	陈洁	陈洁	女	35
20150003	王维	男	4口人	陈洁	陈洁	女	35
20150004	白居易	男	3口人	李丽	李丽	女	32
20150005	刘禹锡	男	4口人	李丽	李丽	女	32
20150006	李清照	女	5口人	王安	王安	男	29
20150007	苏轼	男	2口人	南林	南林	男	34
20150008	屈原	男	4口人	南林	南林	男	34
20150009	陶渊明	男	1口人	王安	王安	男	29

这样以来,就满足了第三范式的要求。

ps:如果把上表中的班主任姓名改成班主任教工号可能更确切,更符合实际情况,不过只要能理解就行。

mysql语法

group by一般用于分组统计,它表达的逻辑就是根据一定的规则,进行分组

select city,count(*) as num from staff group by city;

order by 排序字段,

order by默认采用升序(asc),desc(降序)

如果存在where子句,那么order by必须放到where从句后面。

select province,count(*) as count from college group by province order by count desc limit 1;

limit是mysql的分页函数,可以快速选中我们想要的数

它的用法主要是两种

limit x,表示选中前x条数据

limit x,y表示选中第x+1到x+y条数据,或者理解成从第x+1条数据开始(包括第x+1)往后的y条数据

因此只有limit 15,5表示第16到20的数据

条件查询

1. 条件查询语法

```
SELECT 字段列表 FROM 表名 WHERE 条件列表;
```

2. 条件

符号	功能
>	大于
<	小于
>=	大于等于
<=	小于等于
=	等于
<> 或 !=	不等于
BETWEEN ... AND ...	在某个范围之内(都包含)
IN(...)	多选一
LIKE 占位符	模糊查询 _单个任意字符 %多个任意字符
IS NULL	是NULL
IS NOT NULL	不是NULL
AND 或 &&	并且
OR 或	或者
NOT 或 !	非, 不是

聚合函数

1. 概念:

将一系列数据作为一个整体, 进行纵向计算。

2. 聚合函数分类:

函数名	功能
count(列名)	统计数量(一般选用不为null的列)
max(列名)	最大值
min(列名)	最小值
sum(列名)	求和
avg(列名)	平均值

3. 聚合函数语法:

```
SELECT 聚合函数名(列名) FROM 表;
```

注意: null 值不参与所有聚合函数运算

分组查询

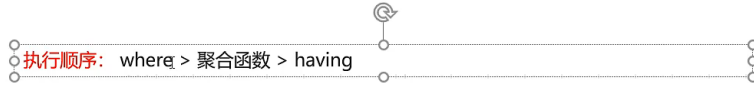
1. 分组查询语法

```
SELECT 字段列表 FROM 表名 [WHERE 分组前条件限定] GROUP BY 分组字段名 [HAVING 分组后条件过滤];
```

注意：分组之后，查询的字段为聚合函数和分组字段，查询其他字段无任何意义

where 和 having 区别：

- 执行时机不一样：where 是分组之前进行限定，不满足where条件，则不参与分组，而having是分组之后对结果进行过滤。
- 可判断的条件不一样：where 不能对聚合函数进行判断，having 可以。



约束名称	描述	关键字
非空约束	保证列中所有数据不能有null值	NOT NULL
唯一约束	保证列中所有数据各不相同	UNIQUE
主键约束	主键是一行数据的唯一标识，要求非空且唯一	PRIMARY KEY
检查约束	保证列中的值满足某一条件	CHECK
默认约束	保存数据时，未指定值则采用默认值	DEFAULT
外键约束	外键用来让两个表的数据之间建立链接，保证数据的一致性和完整性	FOREIGN KEY

Tips：MySQL不支持检查约束

数据结构

二叉查找树(Binary Search Tree, BST)

又叫做二叉排序树、二叉搜索树，是一种对查找和排序都有用的特殊二叉树。

二叉查找树或是空树，或是满足如下三个性质的二叉树：

若其左子树非空，则左子树上所有节点的值都小于根节点的值

若其右子树非空，则右子树上所有节点的值都大于根节点的值

其左右子树都是一棵二叉查找树

一棵平衡二叉树定义(AVL)有如下必要条件：

1. 条件一：它必须是二叉查找树。
2. 条件二：每个节点的左子树和右子树的高度差至多为1。

(四种调整方式)

Nginx

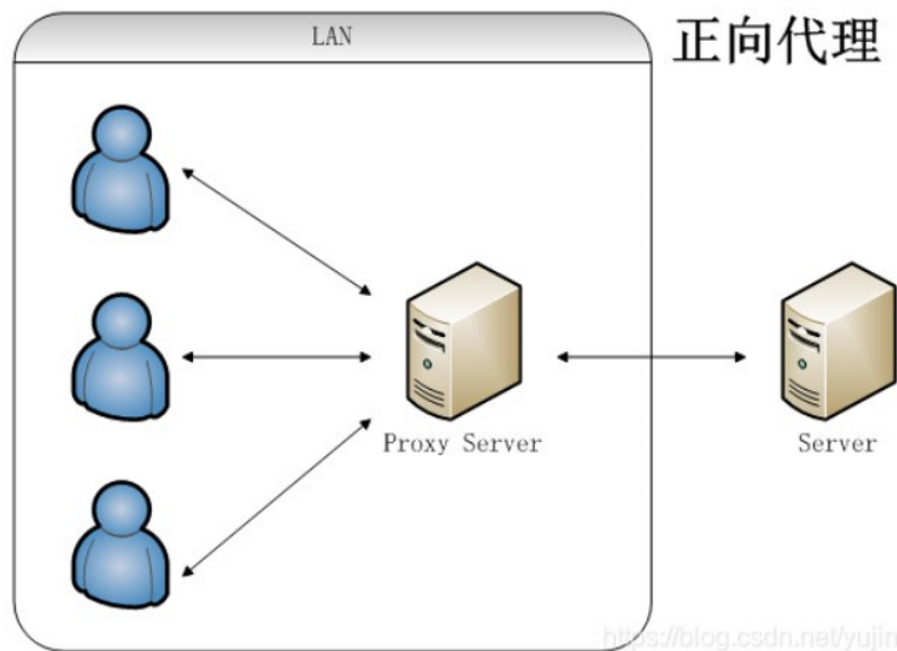
Nginx是一个高性能的HTTP和反向代理服务器，特点是占用内存少，并发能力强，事实上nginx的并发能力确实在同类型的网页服务器中表现较好

nginx专为性能优化而开发，性能是其最重要的要求，十分注重效率，有报告，nginx能支持高达50000个并发连接数

代理

正向代理（代理客户侧网络）

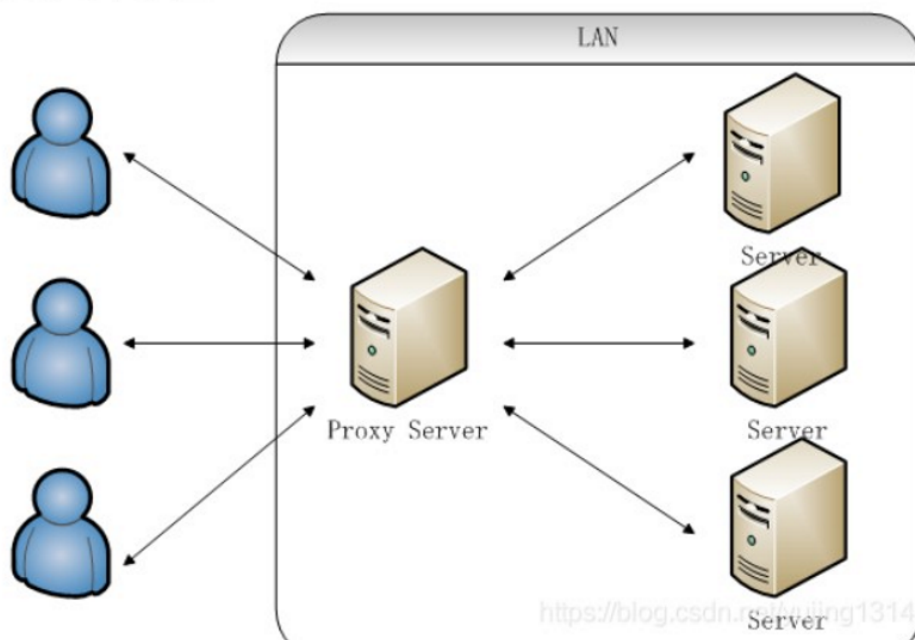
正向代理：局域网中的电脑用户想要直接访问网络是不可行的，只能通过代理服务器来访问，这种代理服务就被称为正向代理



反向代理（代理服务侧网络）

反向代理：客户端无法感知代理，因为客户端访问网络不需要配置，只要把请求发送到反向代理服务器，由反向代理服务器去选择目标服务器获取数据，然后再返回到客户端，此时反向代理服务器和目标服务器对外就是一个服务器，暴露的是代理服务器地址，隐藏了真实服务器IP地址

反向代理



动静分离

为了加快网站的解析速度，可以把动态页面和静态页面交给不同的服务器来解析，加快解析的速度，降低由单个服务器的压力

例如：静态资源存放在Nginx上，动态资源放在Tomcat上

修改nginx.conf

```
server {  
    listen 80;  
    server_name localhost;  
    location = /heroes {  
    }  
}
```

负载均衡

在服务器集群中，Nginx起到一个代理服务器的角色（即反向代理），为了避免单独一个服务器压力过大，将来自用户的请求转发给不同的服务器。

轮询	默认方式
weight	权重方式
ip_hash	依据ip分配方式
least_conn	最少连接方式
fair (第三方)	响应时间方式
url_hash (第三方)	依据URL分配方式

CSDN @ (悟·空)

轮询，权重、ip_hash、最少连接数、响应时间短的优先分配、按访问url的hash结果来分配请求

weight: 权重越高分配到需要处理的请求越多

ip_hash: 这个方法确保了相同的客户端的请求一直发送到相同的服务器

least_conn: 把请求转发给连接数较少的后端服务器, 适合请求处理时间长短不一造成服务器过载的情况。

fair: 按照服务器端的响应时间来分配请求, 响应时间短的优先分配

url_hash: 按访问url的hash结果来分配请求, 使每个url定向到同一个后端服务器, 要配合缓存命中来使用。同一个资源多次请求, 可能会到达不同的服务器上, 导致不必要的多次下载, 缓存命中率不高, 以及一些资源时间的浪费。而使用url_hash, 可以使得同一个url (也就是同一个资源请求) 会到达同一台服务器, 一旦缓存住了资源, 再此收到请求, 就可以从缓存中读取。

Nginx 的原理

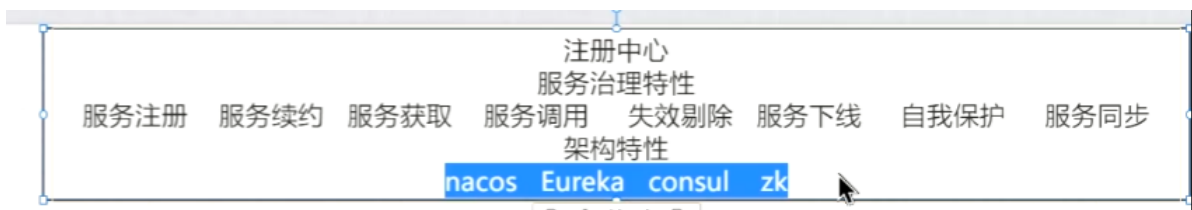
1. master 和 worker

- nginx 启动后, 是由两个进程组成的。master (管理者) worker (工作者)。
- 一个nginx 只有一个master。但可以有多个worker, master管理多个worker工作线程
- 过来的请求由master管理, worker进行争抢式的方式去获取请求

SpringCloud

1、Eureka AP模型 (类似于zookeeper CP模型) (nacos AP/CP Nacos集成了配置中心 (springcloud config) 的功能)

注册 续约 获取 调用 剔除 下线 保护 同步



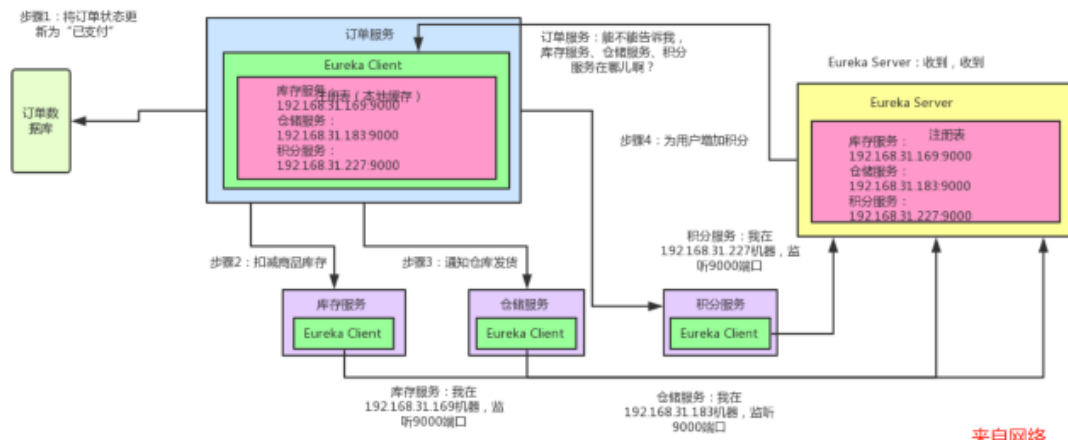
首先考虑一个问题, 订单服务要调用库存服务、仓储服务、积分服务, 如何调用呢?

答: 订单服务根本不知道上述服务在哪台服务器上, 所以没法调用, 而Eureka的作用就是来告诉订单服务它想调用的服务在哪台服务器上, Eureka有客户端和服务端, 每一个服务上面都有Eureka客户端, 可以把本服务的相关信息注册到Eureka服务端上, 客户端完成微服务向Eureka服务的注册与发现。那么我们的订单服务就可以找到库存服务、仓储服务、积分服务了

Eureka客户端: 负责将这个服务的信息注册到Eureka服务端中

Eureka服务端: 相当于一个注册中心, 里面有注册表, 注册表中保存了各个服务所在的机器和端口号, 可以通过Eureka服务端找到各个服务

COW (copy on write) 机制实现注册信息读写分离



2、Feign (类似于dubbo)

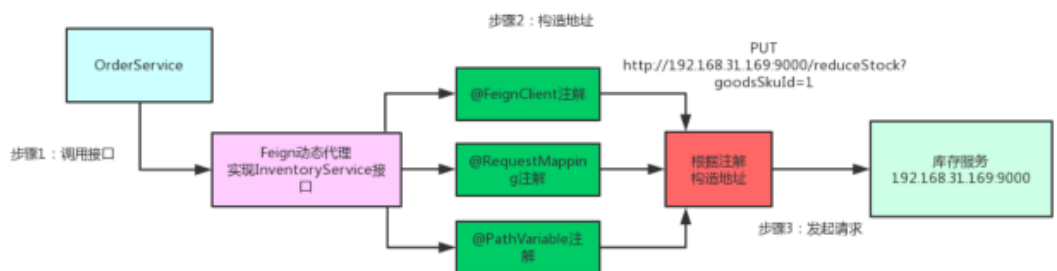
通过上面的Eureka，现在订单服务确实知道库存服务、积分服务、仓储服务在哪了，但是我们如何去调用这些服务呢，如果我们自己去写很多代码调用那就太麻烦了，而SpringCloud已经为我们准备好了一个核心组件：Feign，接下来看如何**通过Feign让订单服务调用库存服务**，注意Feign也是用在消费者端的

首先，如果你对某个接口定义了@FeignClient注解，Feign就会针对这个接口创建一个动态代理

接着你要是调用那个接口，本质就是会调用 Feign创建的动态代理，这是核心中的核心

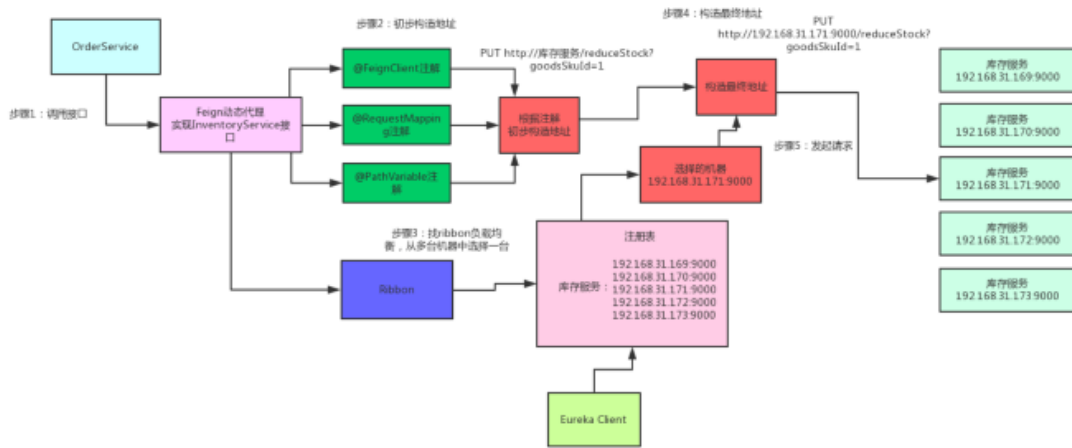
Feign的动态代理会根据你在接口上的@RequestMapping等注解，来动态构造出你要请求的服务的地址

最后针对这个地址，发起请求、解析响应



3、Ribbon (loadbalance)

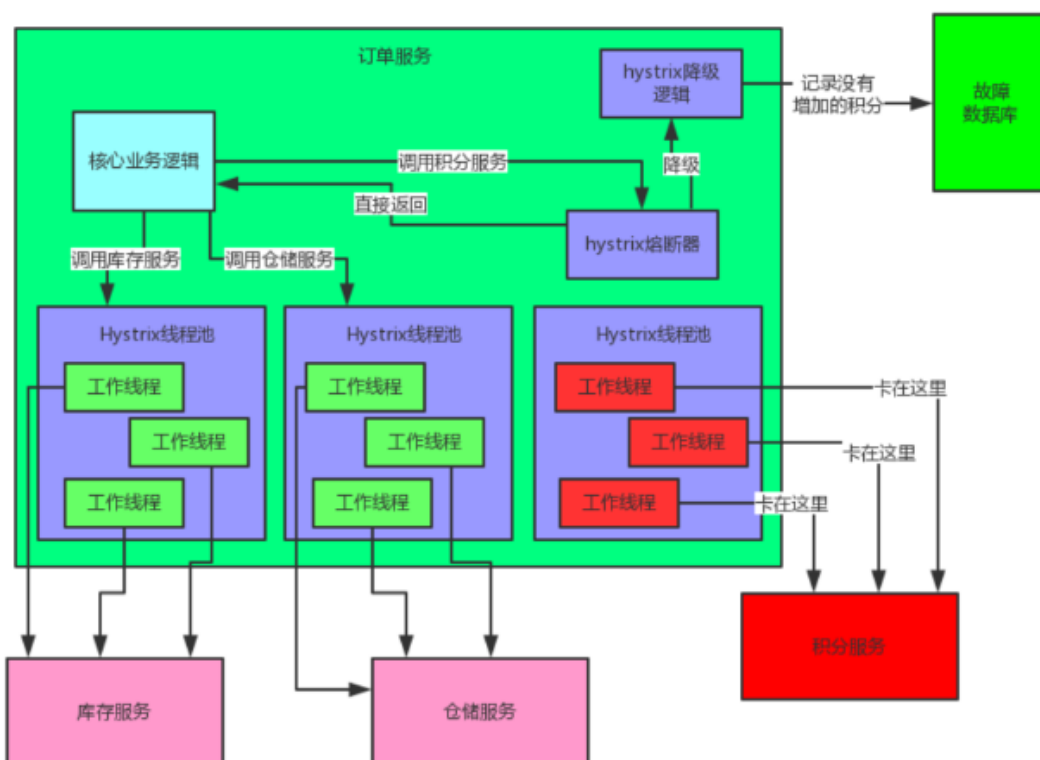
上面可以通过Eureka可以找到服务，然后通过Feign去调用服务，但是如果有多台机器上面都部署了库存服务，我应该使用Feign去调用哪一台上面的服务呢，这个时候就需要Ribbon闪亮登场了，它在服务消费者端配置和使用，它的作用就是负载均衡，然后默认使用的负载均衡算法是轮询算法，Ribbon会从Eureka服务端中获取到对应的服务注册表，然后就知道相应服务的位置，然后Ribbon根据设计的负载均衡算法去选择一台机器，Feign就会针对这些机器构造并发送请求



4、Hystrix

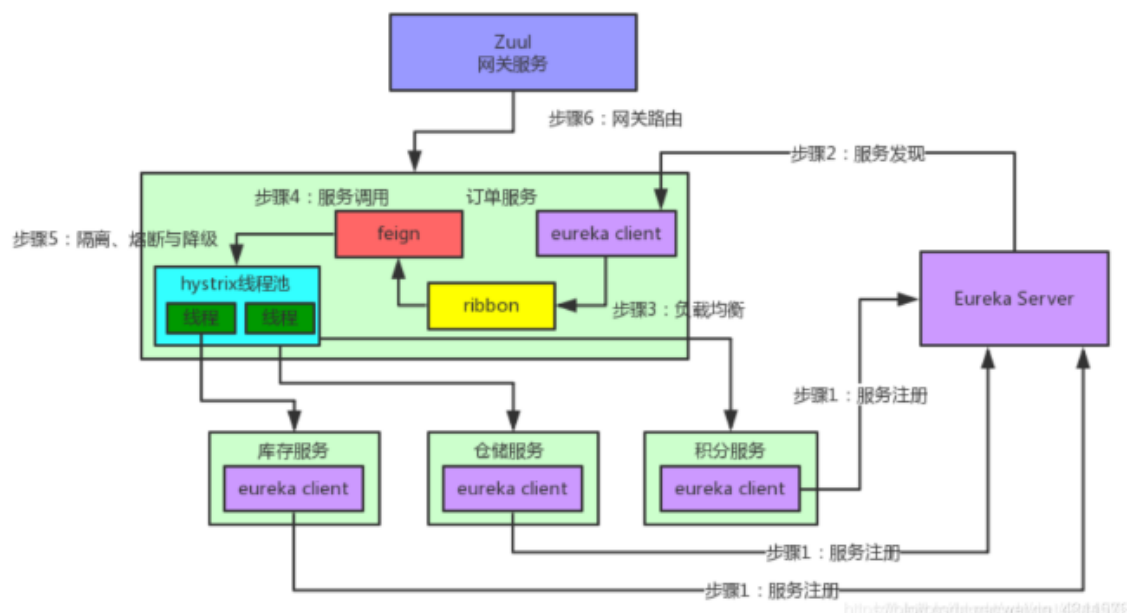
在微服务架构里，一个系统会有多个服务，以本文的业务场景为例：订单服务在一个业务流程里需要调用三个服务，现在假设订单服务自己最多只有100个线程可以处理请求，如果积分服务出错，每次订单服务调用积分服务的时候，都会卡住几秒钟，然后抛出一个超时异常。

即使积分服务挂了，那订单服务也不应该挂掉啊，我们只要让存储服务 and 仓储服务正常工作就可以了，至于积分服务我们后期可以手动给用户加上积分，这个时候就轮到Hystrix闪亮登场了，Hystrix是隔离、熔断以及降级的一个框架，说白了就是Hystrix会搞很多小线程池然后让这些小程序池去请求服务，返回结果，Hystrix相当于是个中间过滤区，如果我们的积分服务挂了，那我们请求积分服务直接就返回了，不需要等待超时时间结束抛出异常，这就是所谓的熔断，但是也不能啥都不干就返回啊，不然我们之后手动加积分咋整啊，那我们每次调用积分服务就在数据库里记录一条消息，这就是所谓的降级，Hystrix隔离、熔断和降级



5、zull (类似于服务器端的nginx)

该组件是负责网络路由的，假设你后台部署了几百个服务，现在有个前端兄弟，人家请求是直接从浏览器那儿发过来的。有一个网关，所有请求都往网关走，网关会根据请求中的一些特征，将请求转发给后端的各个服务。可修改前端展示的url，避免暴露造成危险



MyBatis

半自动化（手写sql）的对象（java对象）关系（数据库关系模型）映射（对应）ORM的持久层框架

（一个Java的Student类，去对应数据库中的一张student表，类中的属性和表中的列一一对应。Student类就对应student表，一个Student对象就对应student表中的一行数据）

namespace: Mapper名称, id: 方法名, namespace+id唯一

缓存关闭, 集群下数据修改, 缓存无法保证一致性

什么是Mybatis

代码 数据库

操作sql 跟我们去操作对象一样对象

ORM框架 Object relational Mapping

怎么去使用

1.config.xml

2.mapper.xml 增删改查

3.mapper.xml对应的namespace 实体对象

初始化工作

1.解析我们的config.xml文件

2.通过解析config.xml我能够得到所有的需要解析的mapper.xml文件

3.解析所有的mapper.xml的文件，并且解析mapper.xml里面的crud的语句标签，并且每个sql语句都会对应一个对象

```
MappedStatement 并且放入Map<String, MappedStatement> mappedStatements
```

Map<String, MappedStatement> mappedStatements key namespace+id(方法名)

4. key为namespace对应的class对象 value MapperProxyFactory工厂

```
Map<Class<?>, MapperProxyFactory<?>> knownMappers = new HashMap<>();
```

创建会话

会去生成执行器 excutor 默认 CachingExecutor 假如 setting里面的二级缓存开关关闭的话，默认是simpleExecutor

语句执行

1.会去从knownMappers 去拿工厂，传进去class对象

代理模式

2.invoke方法

class对象+方法名 去拿 MappedStatement

namespace+id 原来的key 并且namespace+id 不能重复的

MappedStatement 能拿到相关sql语句

代理模式

2.invoke方法

class对象+方法名 去拿 MappedStatement

namespace+id 原来的key 并且namespace+id 不能重复的

MappedStatement 能拿到相关sql语句

CachingExecutor 二级缓存 自己定义缓存策略 创建session的时候就决定了要不要用CachingExecutor

因素来影响

配置 cache

mapper.xml文件里面的配置来决定 cache

select语句

作用域：配置走 namespace 只要配置开启了，不管是什么会话 都是开启的

面试题:

namespace能重复吗? 方法能重复么?

只要保证 namespace+id不重复

同一个mapper方法不能不能重复

假如namespace不一样 能重复

namespace一样, 就算是2个mapper 方法也不能重复

一级二级的缓存 | I

假如 服务是集群的方式的部署的, 怎么做到数据一致性?

其他的服务器进行了update 清除自己本地的缓存 怎么去清除其他机器的缓存

保证不了的

一级二级的缓存? 废物I

假如 服务是集群的方式的部署的, 怎么做到数据一致性?

其他的服务器进行了update 清除自己本地的缓存 怎么去清除其他机器的缓存

|

你们线上一般应该是关闭的

分页有哪些 分页插件怎么实现的?

Spring

Spring是一个轻量级的IOC和AOP容器框架。是为Java应用程序提供基础性服务的一套框架, 目的是用于简化企业应用程序的开发, 它使得开发者只需要关心业务需求。常见的配置方式有三种: 基于XML的配置、基于注解的配置、基于Java的配置。

1.Spring框架是轻量级的JavaEE框架

2.Spring可以解决企业应用开发的复杂性

3.Spring有两个核心部分：IOC和Aop

(1) IOC：控制反转，把创建对象的过程交给Spring进行管理，bean工厂

(2) Aop：面向切面，不修改源代码进行功能增强

轻量: Spring是轻量的，基本的版本大约2MB。

控制反转: Spring通过控制反转实现了松散耦合，对象们给出它们的依赖，而不是创建或查找依赖的对象们。

面向切面的编程(AOP): Spring支持面向切面的编程，并且把应用业务逻辑和系统服务分开。

容器: Spring包含并管理应用中对象的生命周期和配置。

MVC框架: Spring的WEB框架是个精心设计的框架，是Web框架的一个很好的替代品。

事务管理: Spring提供一个持续的事务管理接口，可以扩展到上至本地事务下至全局事务(JTA)。

异常处理: Spring提供方便的API把具体技术相关的异常(比如由JDBC，Hibernate or JDO抛出的)转化为致的unchecked异常。

IOC



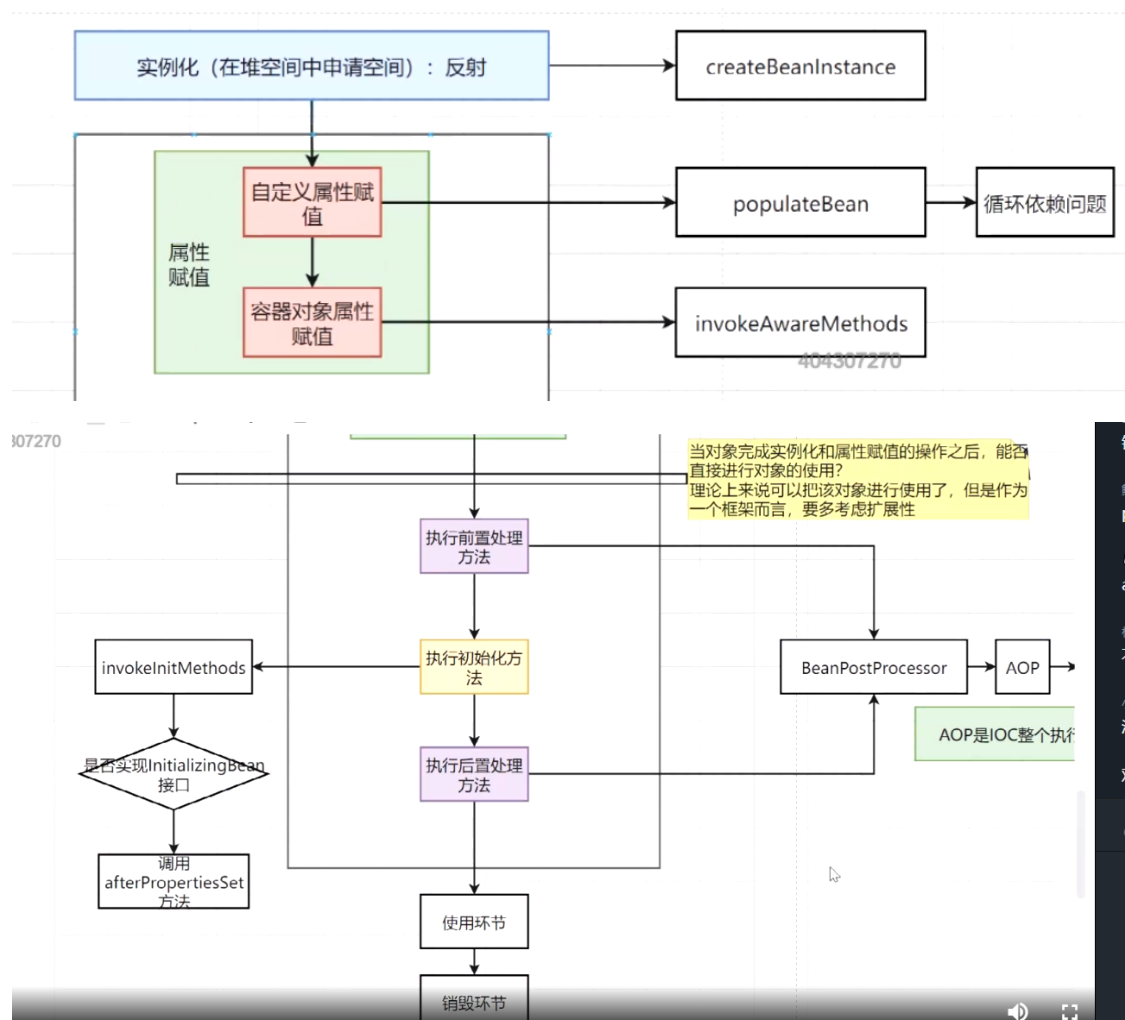
bean生命周期:

实例化 Instantiation (通过反射创建对象)

属性赋值 Populate

初始化 Initialization

销毁 Destruction



bean创建

推断构造方法（默认无参构造，多个构造方法没有@Autowired指定会报错，参数注入byType后byName

) -->普通对象--->依赖注入--->初始化前(@PostConstruct) -->初始化(afterPropertiesSet()) -->初始化后(AOP)--->代理对象=Bean对象

(JVM层面初始化对象调用=方法)

单例bean则放到单例池中，多例bean每次使用时要去创建，不需要放到Map中

- 1.基于xml方式创建对象（bean标签（set、构造），工厂方式）
- 2.基于注解方式的实现（@Bean，@Controller，@Service等等）

什么是自动装配<Bean标签>

根据指定装配规则（属性名称或者属性类型），spring自动将匹配的属值进行注入

将properties文件装配成一个bean加入xml文件中

beanDefinitionReader接口---->BeanDefinition---->bean

postProcessor 后置处理器 BeanFactoryPostProcessor 替换占位符（例如数据源bean对象）

AOP

AOP是IOC执行过程的一个扩展

eg: 根据用户登录做权限检查

//增强的类

@Component

@Aspect //生成代理对象

代理对象继承普通对象，确保类型一致

UserServiceProxy对象--->UserService代理对象--->UserService代理对象.target=普通对象

jdk: 基于接口实现代理

cglib: 没有接口时基于cglib实现代理

spring三级缓冲:

首先看下Spring创建一个bean的基本流程:

创建该实例的原始对象 --> 进行自动装配 --> AOP代理处理 --> 完成bean的创建并加入单例池(即一级缓存)

AService Bean的生命周期

提前AOP

\0. creatingSet<'aService'>

\1. new AService()-->AService普通对象--->三级Map<'aService', lambda<beanName, AService普通对象>>

(如果支持循环依赖则生成三级缓存，可以提前暴露bean)

2. 填充bService属性--->去单例池中找BService-->没有

创建Bservice Bean

2.1. new BService()-->普通对象

2.2.填充aService属性-->单例池-->creatingSet-->循环依赖-->二级-->三级Map-->lambda-->不用AOP--->AService普通对象---二级

--->AOP--->AService代理对象

2.3. 填充其他属性

2.4.?

2.5.单例池Map<" bService", 普通对象>

3, 填充cService属性--->去单例池中找cService-->没有

创建cService Bean

3.1. new CService()-->普通对象

3.2. 填充aService属性--->单例池-->creatingSet-->循环依赖-->二级-->AService对象

3.3. 填充其他属性

3.4.?

3.5. 单例池Map<"cService", 普通对象>

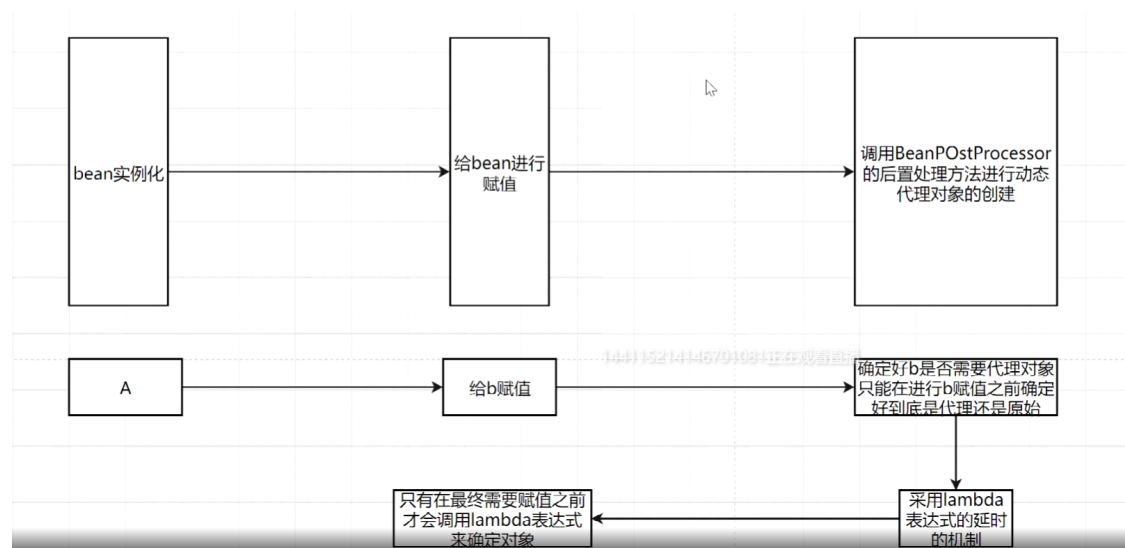
4. 需不需进行AOP---->有没有提前进行AOP? earlyProxyReferences-->AService代理对象

5. 单例池Map<' aService', AService代理对象>

第一级缓存：单例池Map singletonobjects--->

第二级缓存：earlysingletonobjects---> (处理有可能的出现多个不同代理对象问题)

第三级缓存：三级Map singletonFactories 打破循环 lambda--->对象



3. 如果只有两个map，能否解决循环依赖问题？

理论上是可行的，如果想在map中存储两种不同类型的对象，就意味着要添加标识，比如在value中添加一个专门的标志位，但是会带来很多麻烦的操作，比如每次获取对象的时候都要取出来做判断是成品还是半成品，比较麻烦，可以使用两个map轻松解决，而且代码比较优雅

4. 如果只有两个map，能否解决循环依赖问题？

能，但是有前提条件：如果创建的对象中包含了动态代理对象，那么就不能解决，如果没有动态代理，那么可以解决循环依赖的问题。刚刚整个流程中，只有两个地方用到了三级缓存，一个是存储，一个是获取

5. 为什么使用三级缓存就可以解决循环依赖问题？三级缓存是怎么解决循环依赖问题的？

《1》创建代理对象的时候是否需要创建原始对象？

需要，bean的生命周期：先进行实例化操作-->赋值操作-->在BeanPostProcessor的后置处理方法中进行动态代理对象的创建

《2》同一个容器中能否存在同名的不同对象？

不能

《3》如果出现了同名的不同对象之后，要如何处理？

要进行覆盖操作：在getEarlyBeanReference方法中，有代理对象的覆盖操作

《4》为什么添加了三级缓存之后就可以解决这个问题了？

在bean的生命周期中，需要先创建出原始对象，然后进行populateBean的赋值操作，后续在BeanPostProcessor的后置处理方法中进行动态代理对象的生成，但是当出现循环依赖问题的时候，我必须要在对象进行赋值之前确定好要赋值的对象是原始对象还是代理对象，所以只能将动态代理的步骤提前，但是spring是一个跟业务无关的框架，框架无法确定什么时候需要生成代理对象，所以只有在第一次进行属性赋值的时候就唯一性的确定好需要赋值的对象是代理对象还是原始对象。还蒙的同学下去之后进行debug的操作，然后梳理清楚

SPI英文全称为Service Provider Interface，顾名思义，服务提供者接口，它是jdk提供给“服务提供商”或者“插件开发者”使用的接口；

在开发过程中，经常要用到第三方提供的SDK来完成一些业务扩展功能，比如调用第三方的发短信、图片验证码、人脸识别等功能，但问题是，第三方SDK只是提供了标准的功能实现，某些场景下，开发者还想基于这些SDK做一些个性化的定制和扩展，那要怎么办呢？

于是，一些优秀的SDK就通过SPI的机制，将一些接口扩能开放出来，开发者就可以基于这些SPI接口做自身的业务扩展了；

事务

Spring中的事务分为两种方式，一种是编程式事务管理，一种是声明式事务管理。

我们常用的就是声明式事务管理，这是基于AOP实现的事务管理。

声明式事务管理分为XML和注解两种形式。

```
//@Transactional
//事务管理器新建一个数据库连接conn 存到ThreadLocal<Map<DataSource,conn>>中
//conn.autocommit = false
// target.test() UserService普通对象.test() 从ThreadLocal中拿连接执行sql sql1, sql2,
sql3, sql4
//conn. commit() conn.rollback()
```

Spring的事务是如何回滚的？

spring的事务管理是如何实现的？

总: spring的事务是由aop来实现的，首先要生成具体的代理对象，然后按照aop的整套流程来执行具体的操作逻辑，正常情况下要通过通知来完成核心功能，但是事务不是通过通知来实现的，而是通过一个TransactionInterceptor来实现的，然后调用invoke来实现具体的逻辑

- 1、先做准备工作，解析各个方法上事务相关的属性，根据具体的属性来判断是否开始新事务
- 2、当需要开启的时候，获取数据库连接，关闭自动提交功能，开启事务
- 3、执行具体的sql逻辑操作
- 4、在操作过程中，如果执行失败了，那么会通过completeTransactionAfterThrowing看来完成事务的回滚操作，回滚的具体逻辑是通过doRollBack方法来实现的，实现的时候也是要获取连接对象，通过连接对象来回滚
- 5、如果执行过程中，没有任何意外情况的发生，那么通过commitTransactionAfterReturning来完成事务的提交操作，提交的具体逻辑是通过doCommit方法来实现的，实现的时候也是要获取连接，通过连接对象来提交
- 6、当事务执行完毕之后需要清除相关的事务信息cleanupTransactionInfo

SpringBoot

什么是springboot

一种基于spring框架,简化Spring应用开发，能独立运行,自动配置的一种工具框架

springboot的特点（针对web应用的特点）

- o 独立运行的spring项目
- o springboot 创建的web应用内嵌了web服务器
- o springboot提供简化依赖，不需要手动导入依赖包
- o springboot自动配置

beandefinition

自动装配与IOC初始化

什么是SpringBoot???

本质:就是在我们的SpringFramework (IOC AOP) 的基础上进行再封装的上层应用

作用:为了帮助你简化你的开发

复杂的版本SpringMVC 配置文件 依赖 tomcat 简单的版本

约定大于(优于)配置

run() IOC初始化 refresh() 中13个模板方法

@SpringBootApplication

@Target({ElementType.TYPE}) 生效范围

@Retention(RetentionPolicy.RUNTIME)

@Documented 可加入文档

@Inherited 可继承性

元注解 修饰注解的注解

@SpringBootConfiguration

@Configuration

@Indexed 加速索引, 扫描加速

@EnableAutoConfiguration 开启自动装配 扫描加载spring.factories(依赖jar包配置, key-value结构, 其中一个key: EnableAutoConfiguration对应就是我们需要加载的一些配置类)中配置文件到configurations(List)然后过滤筛选(条件筛选是否有bean, 是否有.class文件等)加载所有配置类到spring容器中

@AutoConfigurationPackage 自动配置包

@Import 执行selectImports()方法, 扫描加载spring.factories 导入

@ComponentScan 扫描路径

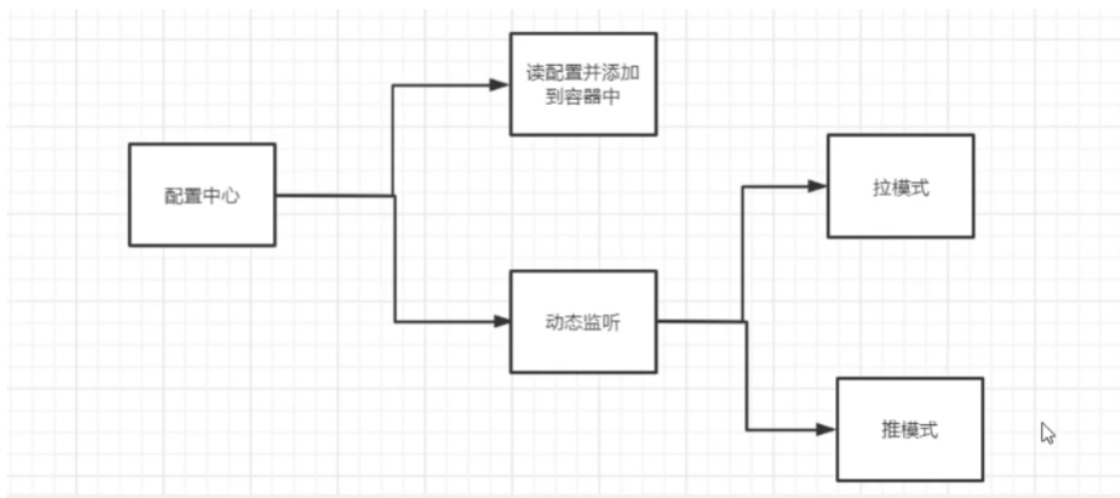
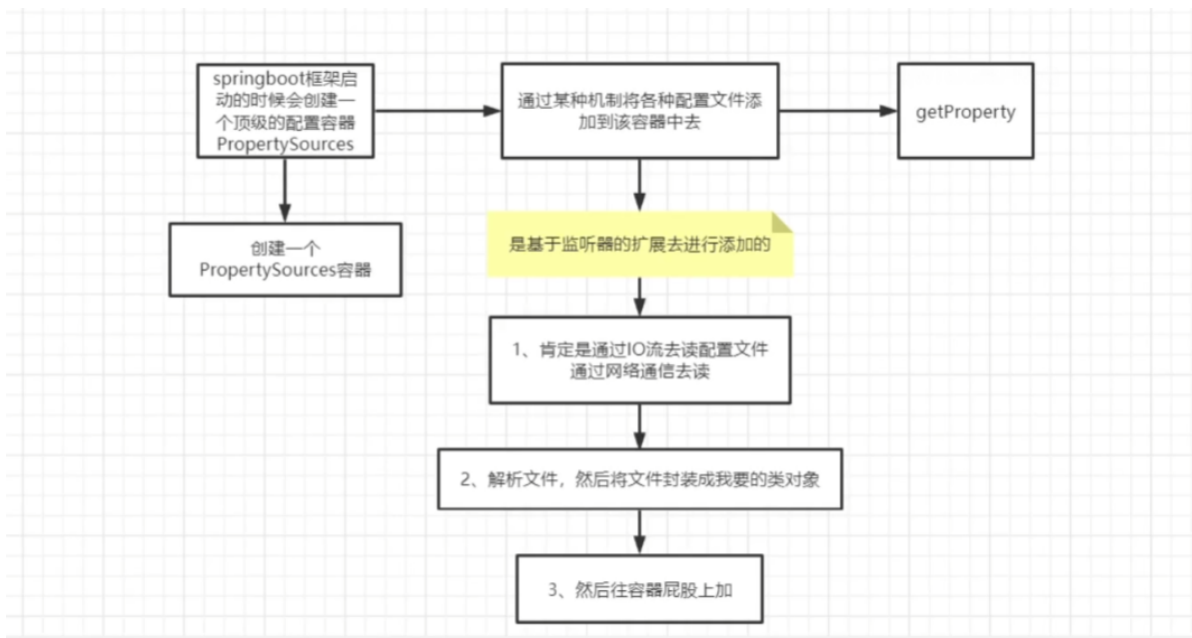
总结:

SpringBoot配置文件加载的顺序是什么??

SpringBoot启动的时候干了什么事?

SpringBoot是如何进行第三方的starter集成的?

首先, 咱们的SpringBoot肯定会执行在咱们的Main方法中的run方法, 而我们的run方法当中会做IOC初始化的操作那么我们的SpringBoot显然不会走配置文件的初始化, 而是注解初始化。很显然, 我会将当前的java配置类的类对象传递进去, 那么我们会走到@SpringBootApplication注解, 接下来, 我们起作用很显然是@EnableAutoConfiguration, 我们的这个注解中会使用到Import注解, 而Import主解注入的类对象一旦实现了ImportSelector接口。那么他起作用的就会是selectImports方法, 而这个方法中, 我们很显然会去加载我们的Spring.factories这个文件, 而我们这个配置文件中会去通过key获取所有的候选对象而我们还会根据auto - configuration.properties去进行条件过滤, 过滤完成之后, 他会在返回的时候加入IOC容器



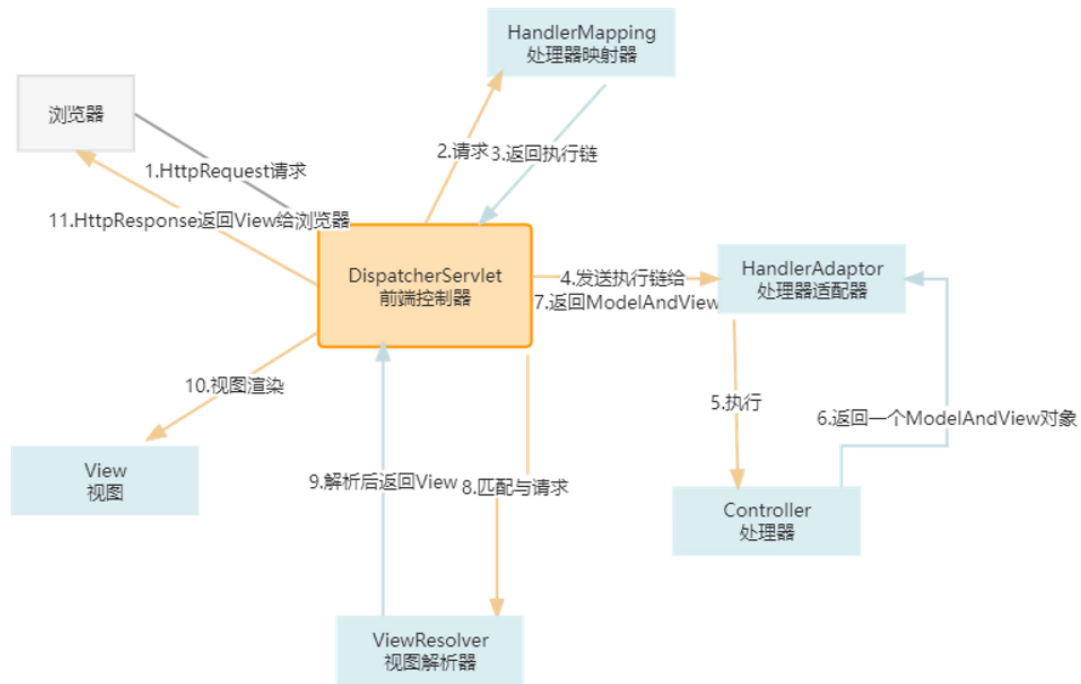
配置中心：拉模式

注册中心：拉与推模式

长轮序

SpringMVC

- 1、用户发送请求至前端控制器DispatcherServlet。
- 2、DispatcherServlet收到请求调用HandlerMapping处理器映射器（接受请求返回执行链）。
- 3、处理器映射器找到具体的处理器(可以根据xml配置、注解进行查找)，生成处理器对象及处理器拦截器(如果有则生成)一并返回给DispatcherServlet。
- 4、DispatcherServlet调用HandlerAdapter处理器适配器。（接收执行链并由Controller执行返回 ModelAndView）
- 5、HandlerAdapter经过适配调用具体的处理器(Controller，也叫后端控制器)。
- 6、Controller执行完成返回 ModelAndView。
- 7、HandlerAdapter将controller执行结果ModelAndView返回给DispatcherServlet。
- 8、DispatcherServlet将ModelAndView传给ViewResolver视图解析器。（处理 ModelAndView 并解析成具体的View）
- 9、ViewResolver解析后返回具体View。
- 10、DispatcherServlet根据View进行渲染视图(即将模型数据填充至视图中)。
- 11、DispatcherServlet响应用户。



图一：SpringMVC工作流程

Tomcat启动，解析web.xml文件，通过listener标签创建父容器

自动创建DispatcherServlet对象

DispatcherServlet对象.init()-->创建spring容器

Redis

Redis 是一个支持nosql（非结构化），key-value数据类型（结构化）、内存、缓存、分布式的、可持久化、非关系型数据库。功能：**缓存、计数器、排行榜、消息队列、分布式锁**

快的原因：

1. 完全**基于内存操作**
2. 命令执行使用**单线程**，避免了线程切换和竞态产生的消耗
3. 基于**非阻塞的IO多路复用机制**（epoll）
4. **C语言实现**，优化过的数据结构，基于几种基础的数据结构，redis做了大量的优化，性能极高
5. kv存储，**跳表**（二分查找，以空间换时间，提高查找效率）

六种淘汰策略

redis内存有限，满了会执行淘汰策略：

1. noeviction(默认策略)：对于写请求**不再提供服务，直接返回错误**（DEL请求和部分特殊请求除外）
2. allkeys-lru：从所有key中使用LRU算法进行淘汰（LRU算法：即**最近最少使用淘汰**算法）
3. volatile-lru：从设置了**过期时间的key**中使用LRU算法进行淘汰
4. allkeys-random：从所有key中**随机淘汰**数据

5.volatile-random: 从设置了过期时间的key中随机淘汰

6.volatile-ttl: 在设置了过期时间的key中, 淘汰过期时间剩余最短的

当使用volatile-lru、volatile-random、volatile-ttl这三种策略时, 如果没有key可以被淘汰, 则和noeviction一样返回错误

Redis过期策略

1. 定期删除, redis默认是每100ms就随机抽取一些设置了过期时间的key, 并检查其是否过期, 如果过期就删除。因此该删除策略并不会删除所有的过期key。
2. 惰性删除, 在客户端需要获取某个key时, redis将首先进行检查, 若该key设置了过期时间并已经过期就会删除。

持久化

Redis 是内存数据库, 它将自己的数据库状态储存在内存里, 如果不想办法将储存在内存中的数据库状态保存到磁盘里, 那么一旦服务器进程退出, 服务器中的数据库状态也会消失不见

Redis 提供了两种持久化方式—— RDB(Redis DataBase) 和 AOF(Append Only File), 这可以将 Redis 在内存中的数据库状态保存到磁盘里

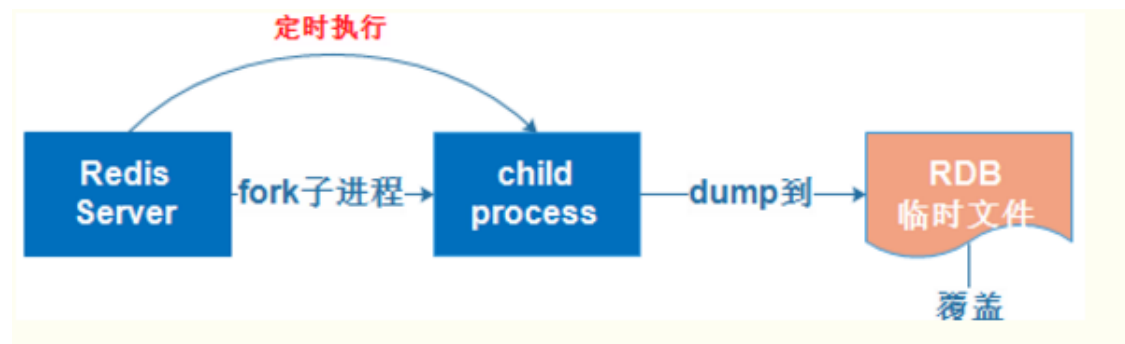
RDB: 在指定的时间间隔能对你的数据进行快照存储。(cow)

RDB持久化是指在指定的时间间隔内将内存中的数据集快照写入磁盘, 实际操作过程是fork一个子进程, 先将数据集写入临时文件, 写入成功后, 再替换之前的文件, 用二进制压缩存储。

RDB 的持久化触发方式有两类: 一类是手动触发, 另一类是自动触发。

手动触发持久化的操作有两个: save (阻塞) 和 bgsave (不阻塞, fork()子线程去执行任务), 它们主要区别体现在: 是否阻塞 Redis 主线程的执行。

自动触发: save m n (指在 m 秒内, 如果有 n 个键发生改变, 则自动触发持久化)

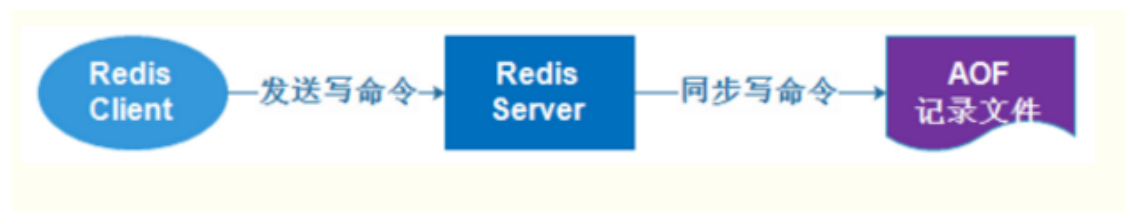


AOF: 记录每次对服务器写的操作,当服务器重启的时候会重新执行这些命令来恢复原始的数据。

AOF持久化以日志的形式记录服务器所处理的每一个写、删除操作, 查询操作不会记录, 以文本的方式记录, 可以打开文件看到详细的操作记录。

AOF 持久化三种策略

- always: 每条 Redis 操作命令都会写入磁盘, 最多丢失一条数据;
- everysec (默认): 每秒钟写入一次磁盘, 最多丢失一秒的数据;
- no: 不设置写入磁盘的规则, 根据当前操作系统来决定何时写入磁盘, Linux 默认 30s 写入一次数据至磁盘。(操作系统根据内存脏写的大小去进行刷盘, 或者定期去进行内存数据刷盘)



AOF文件重写：避免AOF文件愈来愈大

将内存中的数据以命令的方式保存到临时文件中，同时会fork出一条新进程来将文件重写。

重写AOF文件的操作，并没有读取旧的AOF文件，而是将整个内存中的数据库内容用命令的方式重写了一个新的AOF文件

- 如果数据不能丢失，RDB和AOF混用
- 如果只作为缓存使用，可以承受几分钟的数据丢失的话，可以只使用RDB。
- 如果只使用AOF，优先使用everysec的写回策略。

高频数据放在缓存中，因为缓存是直接存储在内存中，访问的速度快。使用缓存对于数据库能够承受的并发访问数量远远大于直接访问数据库，从而提高了整个系统的并发量

一旦涉及大数据量的需求，比如一些商品抢购的情景，或者是主页访问量瞬间较大的时候，单一使用数据库来保存数据的系统会因为面向磁盘，磁盘读/写速度比较慢的问题而存在严重的性能弊端，一瞬间成千上万的请求到来，需要系统在极短的时间内完成成千上万次的读/写操作，这个时候往往不是数据库能够承受的，极其容易造成数据库系统瘫痪，最终导致服务宕机的严重生产问题。

- 缓存击穿：key对应的数据存在，但在redis中过期，此时若有大量并发请求过来，这些请求发现缓存过期一般都会从后端DB加载数据并回设到缓存，这个时候大并发的请求可能会瞬间把后端DB压垮。(cache-miss)
- 缓存穿透：key对应的数据在数据源并不存在，每次针对此key的请求从缓存获取不到，请求都会到数据源，从而可能压垮数据源。比如用一个不存在的用户id获取用户信息，不论缓存还是数据库都没有，若黑客利用此漏洞进行攻击可能压垮数据库。
- 缓存雪崩：当缓存服务器重启或者大量缓存集中在某一个时间段失效，这样在失效的时候，也会给后端系统(比如DB)带来很大压力。

有很多种方法可以有效地解决缓存穿透问题，最常见的则是采用布隆过滤器，将所有可能存在的数据哈希到一个足够大的bitmap中，一个一定不存在的数据会被这个bitmap拦截掉，如果不存在则直接返回false，从而避免了对底层存储系统的查询压力。另外也有一个更为简单粗暴的方法（我们采用的就是这种），如果一个查询返回的数据为空（不管是数据不存在，还是系统故障），我们仍然把这个空结果进行缓存，但它的过期时间会很短，最长不超过五分钟。

```
if (cacheValue != null) {
    return cacheValue;
} else {
    //数据库查询不到，为空
    cacheValue = GetProductListFromDB();
    if (cacheValue == null) {
        //如果发现为空，设置个默认值，也缓存起来
        cacheValue = string.Empty;
    }
    CacheHelper.Add(cacheKey, cacheValue, cacheTime);
    return cacheValue;
}
```

缓存击穿解决方案

- 1、**热数据，设置永远不过期**。
- 2、**加锁对请求进行同步**。锁内逻辑：再次查询缓存，查不到转查数据库并且进行数据缓存，后面的请求就直接缓存，也避免再次查数据库。（**DCL双重检查锁**）

SETNX，是「SET if Not eXists」的缩写，也就是只有不存在的时候才设置，可以利用它来实现锁的效果。

缓存雪崩解决方案

- 1.保持缓存层的**高可用性**

使用Redis 哨兵模式或者Redis 集群部署方式，即便个别Redis 节点下线，整个缓存层依然可以使用。除此之外，还可以在多个机房部署 Redis，这样即便是机房死机，依然可以实现缓存层的高可用。

- 2.**限流降级**组件

无论是缓存层还是存储层都会有出错的概率，可以将它们视为资源。作为并发量较大的分布式系统，假如有一个资源不可用，可能会造成所有线程在获取这个资源时异常，造成整个系统不可用。降级在高并发系统中是非常正常的，比如推荐服务中，如果个性化推荐服务不可用，可以降级补充热点数据，不至于造成整个推荐服务不可用。常见的限流降级组件如 Hystrix、Sentinel 等。

- 3.缓存不过期

Redis 中保存的 key 永不失效，这样就不会出现大量缓存同时失效的问题，但是随之而来的就是 Redis 需要更多的存储空间。

- 4.**优化缓存过期时间(不同过期时间)**

设计缓存时，为每一个 key 选择合适的过期时间，避免大量的 key 在同一时刻同时失效，造成缓存雪崩。

- 5.使用**互斥锁**重建缓存

在高并发场景下，为了避免大量的请求同时到达存储层查询数据、重建缓存，可以使用互斥锁控制，如根据 key 去缓存层查询数据，当缓存层为命中时，对 key 加锁，然后从存储层查询数据，将数据写入缓存层，最后释放锁。若其他线程发现获取锁失败，则让线程休眠一段时间后重试。对于锁的类型，如果是在单机环境下可以使用 Java 并发包下的 Lock，如果是在分布式环境下，可以使用分布式锁（Redis 中的 SETNX 方法）。

分布式环境下使用Redis 分布式锁实现缓存重建，优点是设计思路简单，对数据一致性有保障;缺点是代码复杂度增加，有可能会造成用户等待。假设在高并发下，缓存重建期间 key 是锁着的，如果当前并发 1000 个请求，其中 999 个都在阻塞，会导致 999 个用户请求阻塞而等待。

6.异步重建缓存

在这种方案下构建缓存采取异步策略，会从线程池中获取线程来异步构建缓存，从而不会让所有的请求直接到达存储层，该方案中每个Redis key 维护逻辑超时时间，当逻辑超时时间小于当前时间时，则说明当前缓存已经失效，应当进行缓存更新，否则说明当前缓存未失效，直接返回缓存中的 value 值。如在Redis 中将 key 的过期时间设置为 60 min，在对应的 value 中设置逻辑过期时间为 30 min。这样当 key 到了 30 min 的逻辑过期时间，就可以异步更新这个 key 的缓存，但是在更新缓存的这段时间内，旧的缓存依然可用。这种异步重建缓存的方式可以有效避免大量的 key 同时失效。

Redis支持的各种功能

- \1. 缓存,提升系统访问数据的性能(减少了层级之间的调用)
- \2. 低配版本的消息队列
- \3. 实现各种逻辑锁
- \4. session共享使用redis解决

Redis具备五种数据类型，每种数据类型都是针对key-value结构中的value而言。

String 普通数据的缓存

Hash 存放对象，应用场景：存放游客的购物车

List 秒杀

Set 兴趣爱好；关注，粉丝，共同好友

Zset 排行榜

Redis是单实例,单线程的技术，所以要部署集群（redis.conf）

Redis分布式集群（Redis Cluster）：**主从复制，哨兵监听**（AP模型，，redis不保证数据一致性）
（读写不一致问题）只能保证最终一致性

集群节点两两互联,底层依赖二进制传输协议的,优化传输速度，内部之间传输很快.(集群的基础)

集群的监控监听机制,交给主节点投票(哨兵集成到了集群的主节点中),任意的节点宕机,只要剩下的主节点满足过半.

客户端连接任意一个节点即可

哨兵监听：每1秒钟发起一次心跳检测(rpc远程通信协议),一旦发现主节点宕机,哨兵集群发起投票选举,最终过半的票数确定事件的结果

槽道管理：将16384个槽道划分给各主节点，每个key存储在相应槽道中

cluster集群

多主多从

哨兵模式

一主多重，投票自动拉取

Redisson

RedisTemplate

分布式锁

redis是KV结构，判断可以是否存在，不存在则说明没有人在使用，同时（executo() 方法执行脚本）lua脚本保证原子性（事务无法实现判断逻辑，无法根据上一个结果执行下一步操作）

这个逻辑是原子的：

- 1.setnx 实现原子 指令执行单线程 只能针对String 可能要用到其他的数据类型来实现分布式锁 不支持
- 2.lua脚本 保证多个指令的原子性，并且能拿到中间指令的执行结果作为后续判断。为什么不用事务去保证原子性。

1.假如我要实现一把重入锁怎么办？

减少死锁 同一个线程加锁多次，我要支持

必要条件：

- 1.能做锁 不同线程的互斥 key
- 2.记录线程信息，判断当前加锁的线程是不是现在的线程 filed 记录当前加锁是哪个线程
- 3.记录重入次数 value

数据可以去实现 hash

- 1.判断下这个小key是不是跟我当前线程相同 如果相同 value+1

2.假如时间过期了，但是业务没执行完怎么办？

时间加上去 续期

定期去检查 如果这个锁还存在 这个锁还没释放 是不是还在做业务逻辑 可以去续期

释放了我就不续期了

定时调度 不能一直调度 必须这个key还存在

看门狗

时间轮 (hashWheelTimer)

红锁：数量过半加锁成功

幂等

多次请求所产生的影响与一次请求执行的影响效果相同

转账超时重试是否会多转一笔？

幂等控制：即可以保证转账正常进行，又可以保证不会多转一笔

方案一

就是下游系统(转账系统)提供一个对应的查询接口。如果接口超时了，先查下对应的记录，如果查到是成功，就走成功流程，如果是失败，就按失败处理

方案二

下游接口支持幂等，上游系统如果调用超时，发起重试即可

幂等设计：全局唯一ID（UUID，雪花算法是一种生成分布式全局唯一ID的算法）

根据全局唯一ID（唯一索引）查询数据库，如果数据存在，就拦截重复请求（抛异常，返回执行成功），不存在执行重复请求

防重主要为了避免产生重复数据，把重复请求拦截下来即可。而幂等设计除了拦截已经处理的请求，还要求每次相同的请求都返回一样的结果。不过呢，很多时候，它们的处理可以是类似，只是返回响应不一样

项目

mycat

mysql实现了主从数据自动同步，如果主服务器故障，应用程序怎么切换到非故障服务器？mysql不会自动切换；要中间件做。

mycat：数据库中间件（不是一个数据库）；实现高可用（解决数据量太大，高并发等问题），功能包括：主从自动切换（双机热备）；读写分离（负载均衡）；数据分库分表等；

我们现在普遍的Java应用程序都是直接连接了MySQL软件进行读写操作，也就是我们在Java中的配置文件等定义了mysql的数据源，直接连接到了我们的mysql软件，但是当某些情况下我们可能需要用到了多个数据库，这个时候我们可能就需要配多个数据源去连接我们的多个数据库，这个时候我们进行sql操作的时候就会很麻烦，因为Java与数据库有了一个紧密的耦合度，但是如果我们在Java应用程序与mysql中间使用了mycat，我们只需要访问mycat就可以了，至于数据源等问题，mycat会直接帮我们搞定。

再来说一下高访问量高并发，我们都知道mysql数据库实际上在数据查询上是有一个瓶颈的，当我们的数据太多的时候，已经互联网上有高并发的请求的时候，这个时候对我们mysql的压力是非常大的，当访问量一大，就可能会出现查不出数据，响应的时间太长等，这个时候我们可能需要有多个服务器对数据库进行读写分离，以及对数据库进行集群，这个时候我们的sql语句要进行分类，哪个sql语句要访问哪个数据库，这个时候只要交给mycat就可以了。

3.mycat的原理

实现的是对客户端发起的sql访问做拦截.计算之后开启后端数据库的连接客户端执行具体的sql,根据不同的数据库类型内部拦截后重新组织sql语句()

server.xml: 管理者mycat的逻辑库、表、分片规则、DataNode以及DataSource。

schema标签用于定义mycat实例中的逻辑库.中 部署双机热备，宕机切换

schema.xml: 保存了所有mycat需要的系统配置信息。mycat和真实数据库的 (mysql, oracle) 连接

rule.xml里面就定义了我们表进行拆分所涉及到的规则定义

mysql

Mysql主从复制 (bin日志实现, 记录数据库的所有sql操作)

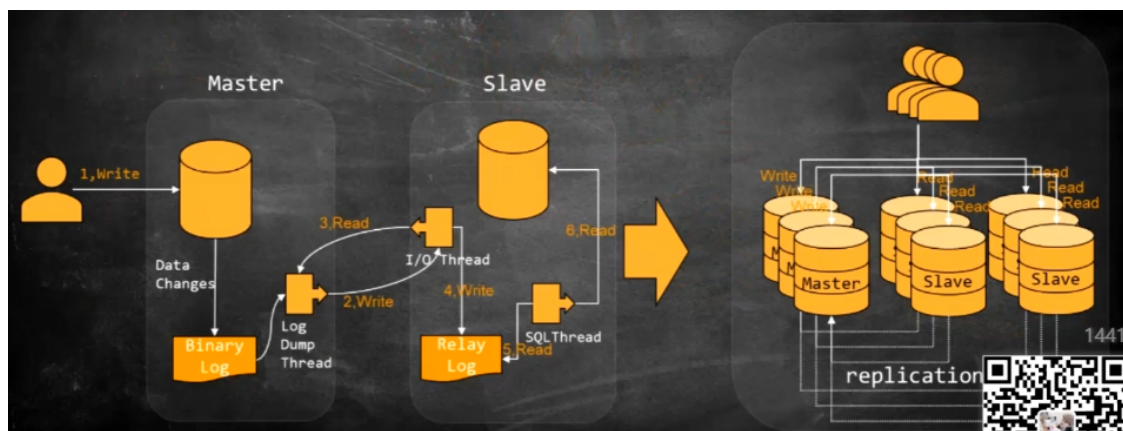
主数据库实时备份到从数据库

作用: 实时备份 (热备份); 数据服务器出问题, 可以实时切换到从数据库。

原理: 主: 写入; 从: 从主服务器中实时得到数据, 写到从服务器。开启log-bin:记录所有变化

读写分离: 主数据库进行增改查操作, 从数据库进行查询操作 (读操作多, 写操作少, 并发量大)

binlog IOThread RelayLog SQLThread



阿里的Druid数据库连接池 (直接添加依赖): Druid连接池是阿里巴巴开源的数据库连接池项目。**Druid连接池为监控而生**, 内置强大的监控功能, 监控特性不影响性能。功能强大, **能防SQL注入**, 内置Logging能诊断Hack应用行为。

Maven

Maven是一个**项目依赖管理工具**, 它包含了一个对象模型。一组标准集合, 一个依赖管理系统。和用来运行定义在生命周期阶段中插件目标和逻辑。

核心功能

Maven的核心功能是合理叙述项目间的依赖关系, 通俗点 就是通过pom.xml文件的配置获取jar包不用手动的去添加jar包(自动从仓库引入数十个甚至上百个jar包, 也可解决jar包版本冲突问题),

仓库分为: 本地仓库, 第三方仓库 (私服), 中央仓库

生命周期: 清理、编译、测试、报告、打包、安装、部署

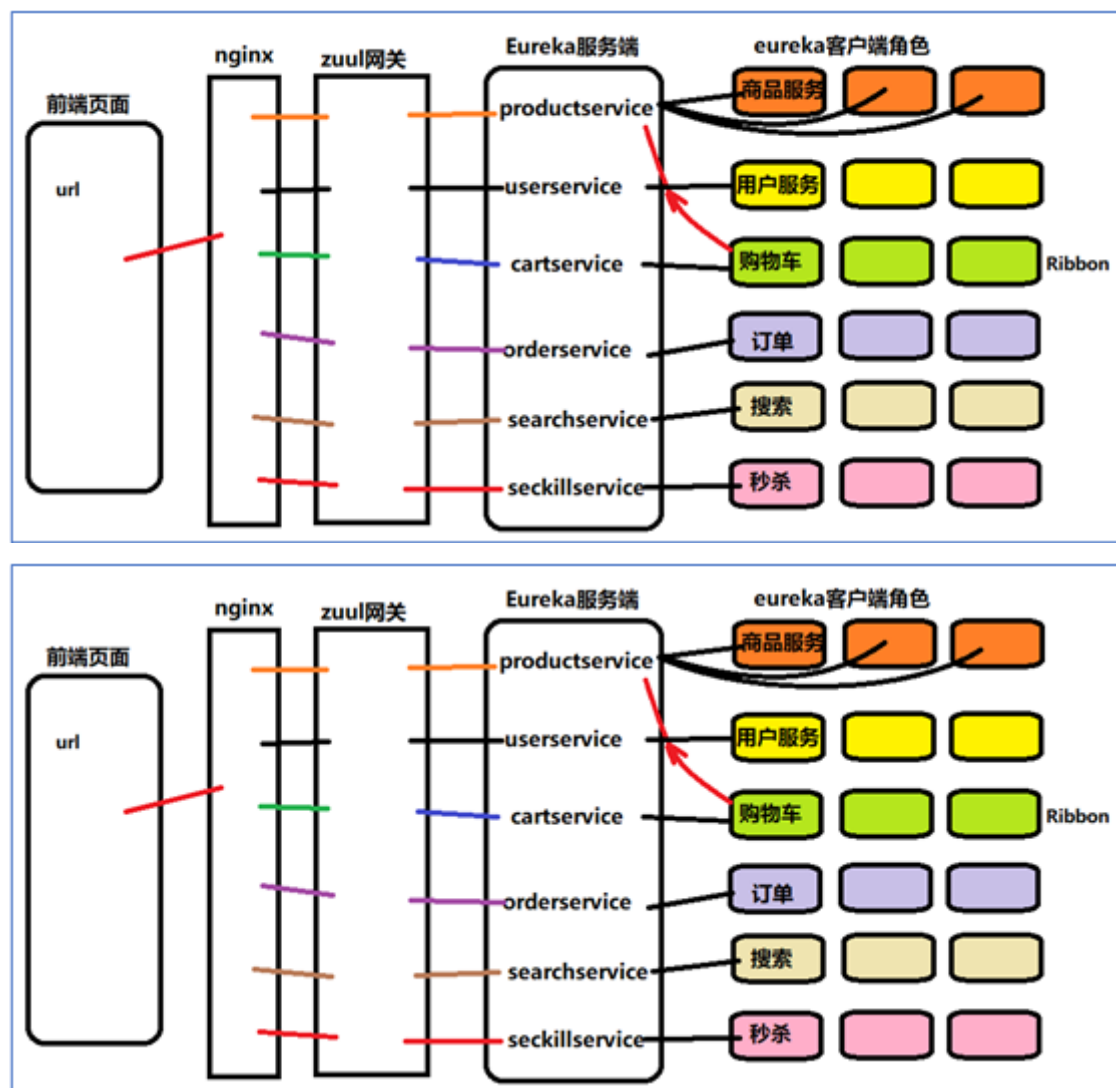
系统拆分

横向拆分：控制层，业务层，持久层

纵向拆分：业务功能拆分，解决功能强耦合性

单体系统——>系统纵向拆分（微服务）

单体系统(所有功能都是启动在一个web中)



nginx监听接受请求，根据匹配规则转发，转发到zuul网关，zuul网关根据路由匹配再转发请求，经过ribbon负载均衡找到具体服务器

模块

一个持久层简化依赖工程

继承parent，依赖jdbc,mysql,mybatis。

作用：提供持久层的相关依赖给其他功能子系统，完成持久层依赖的传递功能

公用的工具工程

在系统开发过程中,多个系统可能用到同一个工具类,这种也叫作共享资源,可以单独提取一个维护工具类,视图类,pojo类的common-resource工程。

product服务工程 （查询功能等）

商品新增服务工程（图片上传功能）

user服务工程（注册功能、登录功能）（判断用户是否存在，保存用户）

公用工程common-rediscluster

购物车服务工程

Ajax

- AJAX 是与服务器交换数据并更新部分网页的艺术，在不重新加载整个页面的情况下。
- 是一个 默认异步 执行机制的功能

Springboot整合JedisCluster操作集群

(redis缓存，判断缓存是否存在这个商品数据，如果存在,读取出来.直接封装返回,不再调用持久层、如果不存在,调用持久层获取数据,并存储一份在redis供后续使用。user登录通过本地cookie数据（EM_TICKET）查询redis，用户在首次登录将用户数据存储在redis（Redis分布式集群））

分布式锁的概念（可在根据业务场景细分读写锁）

分布式锁其实可以理解为：控制分布式系统有序的去对共享资源进行操作，通过互斥来保持一致性。

举个例子：假设共享的资源就是一个房子，里面有各种书，分布式系统就是要进屋看书的人，分布式锁就是保证这个房子只有一个门并且一次只有一个人可以进，而且门只有一把钥匙。然后许多人要去看书，可以，排队，第一个人拿着钥匙把门打开进屋看书并且把门锁上，然后第二个人没有钥匙，那就等着，等第一个出来，然后你在拿着钥匙进去，然后就是以此类推。

锁存在的意义

原因其实很简单：因为我们想让同一时刻只有一个线程在执行某段代码。

因为如果同时出现多个线程去执行，可能会带来我们不想要的结果，可能是数据错误，也可能是服务宕机等等。

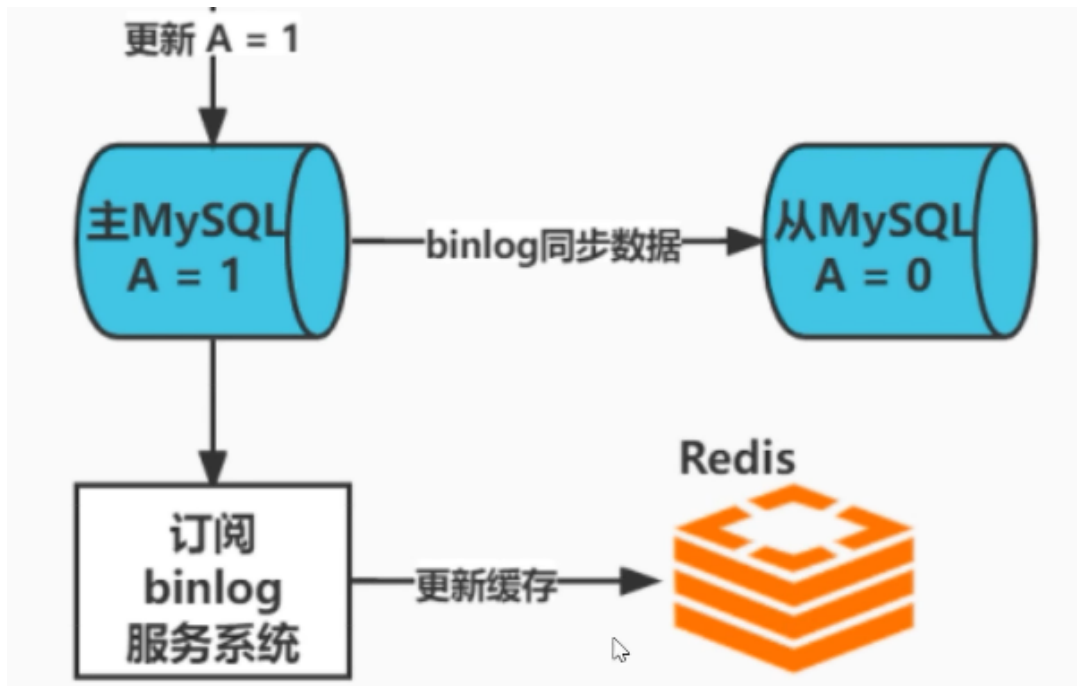
数据不一致

高并发锁的使用

redis缓存需要和数据库数据保持一致，只要商家对商品进行修改/删除,就将缓存数据删除;高并发下（先删redis，再改数据有时间差）还是可能查到旧数据（mysql中旧数据）加到redis（白删除情况），所以在redis中对要更改id数据加锁，不允许加载到redis缓存中，更改完后释放锁（**双写不一致问题**）

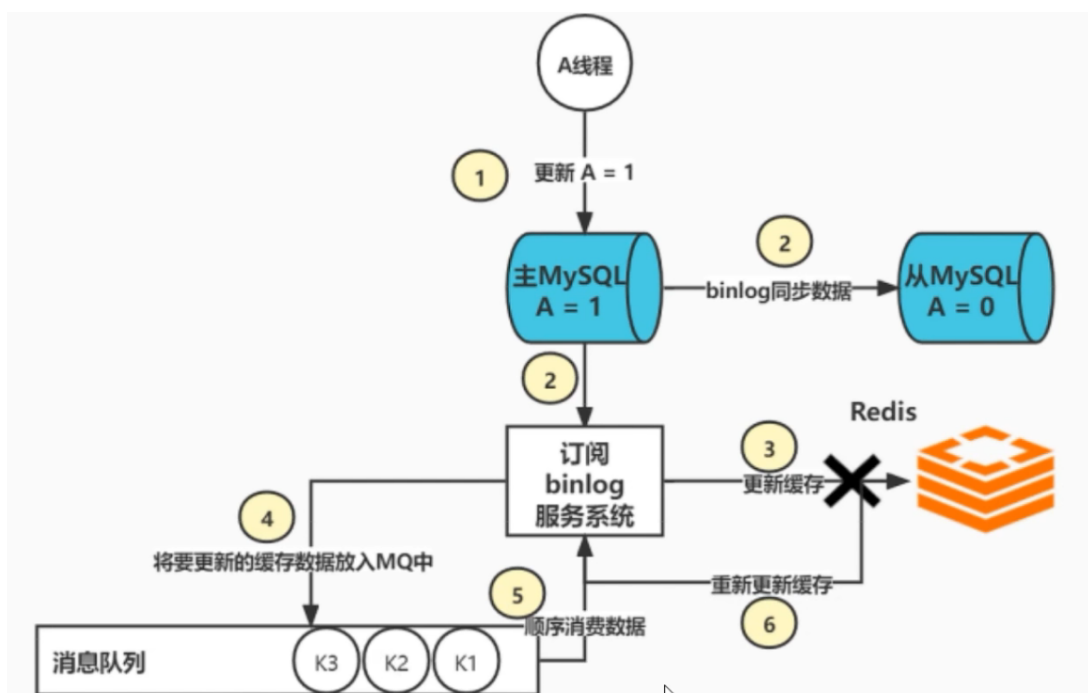
- 1、删缓存，更新数据库，再延迟（让其他线程可以把旧数据写入缓存）删除缓存（**延迟双删**）

2、canal binlog (根据binlog更新缓存, 拿到最新数据, 避免mysql读写分离从读库查到不一致数据, 或者redis的更新只从主库中更新) (canal把自己伪装成mysql 从库, 向主库发送dump协议, 请求binlog数据)



3、加队列, 串行化执行

4、加锁



SETEX key seconds value (SET key value)

将值 value 关联到 key , 并将 key 的生存时间设为 seconds (以秒为单位)。

如果 key 已经存在, SETEX 命令将覆写旧值。

缓存未命中: 两个Tomcat线程访问redis, 没有缓存数据, 去拿redis分布式锁, 拿到的去查DB, 添加redis缓存, 另一个自旋一定时间 (或休眠) 重新查redis

SETNX key value (分布式锁、redisson、lock底层也是setnx)

将 key 的值设为 value，当且仅当 key 不存在。

若给定的 key 已经存在，则 SETNX 不做任何动作

登录超时功能：

用户的ticket的值一直保存在redis中。登录状态一直存在，不符合业务逻辑，必须设置登录超时时间，但不能写死，制定一个阈值，用户登录后不断的操作，一旦时间到达这个阈值，就续租；一旦发现某次访问redis后.ticket剩余时间少于10分钟,自动给延长5分钟

登录顶替：当一个账号在另外一个地方登录时，直接清除前面的redis中登录状态，创建新的登录状态。

秒杀实现：

静态资源 (cdn)

1、单个商品，限流熔断，砍掉大量请求，放行小量请求都后台（奇偶页面，Nginx拦截过滤恶意请求刷单，Nginx累加请求数，达到一定数量不在接受请求），单体系统即可，**接入层过滤**

2、京东淘宝类，无法砍掉大量请求（多品交易，一个订单多个商品），三高（高并发、高可用、高性能），微服务架构（分布式锁）

3、直播间（单品交易（商品种类不同），一个订单一个商品，库存有限），任意某一时间并发量高，倾斜热点（累加器识别，做缓存，防止数据库压力大），非热点数据到库访问，秒杀，在线运营（根据某一时刻商品请求猛增，识别热点商品，加缓存，做过滤限流熔断，放行一定量请求），实际到后台的请求量不多，不能接入层过滤请求，商品上架下架不确定，也会有不同商品下单请求

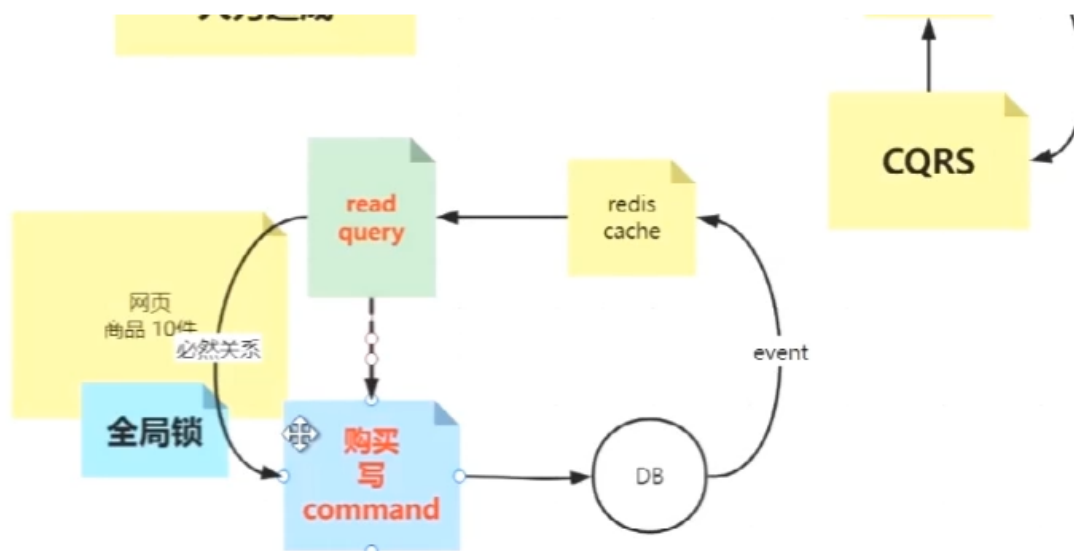
限流算法：**令牌桶，计数器，漏桶、滑动窗口**

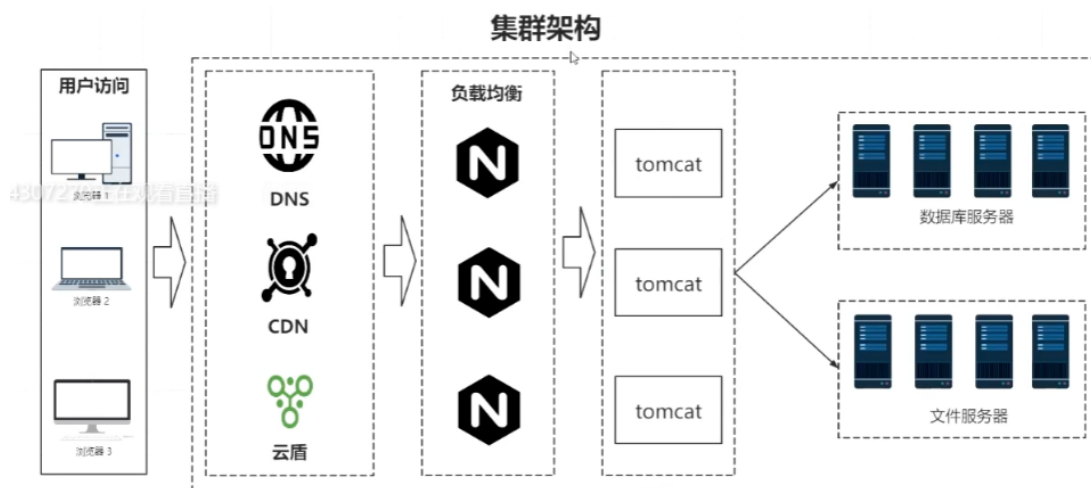
请求链路过长：阻塞线程，qps急剧下降，吞吐量下降（异步拆解）

CQRS读写分离架构

Command包括增、删、改。

Query只包含查





工作经历

主页、文件分析页、部门分析页、用户分析页、风险部门、风险日志、权限管理

数据分析平台去年十月开发，已经迭代两个版本，**数据管理，业务逻辑复杂**（整个数据处理过程都是批处理，要确保低延迟），核心功能**监控所有数据文件的流转**

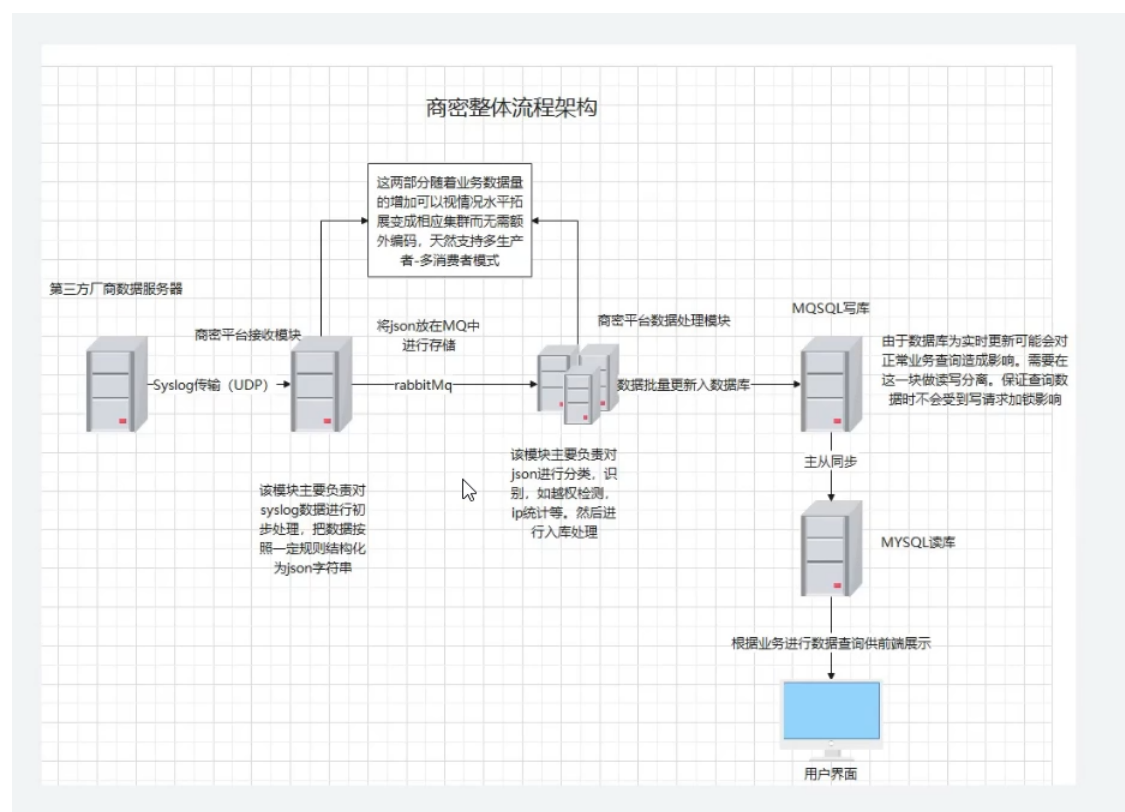
（告警实例：某用户批量导出系统数据提出告警，根据文件本身机密程度做不同告警规则处理，跨部门文件流转，跨岗位文件流转，跨项目组文件流转进行警告）

服务器：**16核32G ssd硬盘** **基本没什么数据积累问题**

对不同设备（主要是**日志文件**）进行**数据采集**（ftp（四五个G/天），syslog（客户端，udp方式接收，几百M/天），**离线上传（主要是网盘）**），接收后数据落盘（写磁盘，防丢失），加工，然后解析做序列化同一结构化成json格式，传递到MQ，后传到**数据处理模块**（业务逻辑操作（识别（日志同一个用户识别匹配），ip，时间段等标识统计访问信息，文件更新操作（归属分类，交互关系，风险统计，溯源，存储位置），审计信息等），**入库**（5张基础底表），**分布式，集群**）

当前面的步骤没问题后，代表着**数据整理完已经正常入库**。这时候需要调用**异步任务执行数据分析处理**。如下图，scanner和线程池组成了另一个消费-生产模型。需要注意的是，为了确保数据不被重复消费，永远**只能有一个scanner进行数据库扫描**（置位需要时间，如果置位的过程中存在另一个scanner，会导致某条数据会被拿出来消费两次进而导致数据计算不准确）。

数据库读写分离：测试样本的数据量200-300万行，实际上数据到了五千万，一年可达上亿



SQL优化问题是一个比较复杂的问题，并不是说出现慢sql之后才开始进行优化，一般在进行数据库设计的时候就会考虑到优化的问题，比如会从mysql的整体架构，表字段的类型设计，表schema的设计来考虑，当然这些工作只是最开始的预防，后续的时候还会出现问题，那么这个时候会从索引、sql语句、性能监控，配置参数多个维度来考虑优化问题，当然使用最多的还是索引和sql语句的优化，比如我在项目中，我们就出现了一个慢sql的问题，表的数据量并不大，大概有两百七十万条记录，就是两个表联合查询问题，但是耗费时间大概是4s，这个在我们线上环境是不允许的，我通过执行计划发现这条sql语句也用到了索引，但是效率依然极低，通过具体的分析发现可能是大量的回表造成的，所以当时做了一个组合索引，然后发现执行结果变成了0.05秒，提升了很多，这是我之前接触过的印象比较深刻的sql优化

多线程（FutureTask）、定时任务、线程池、表设计，redis

分库分表缺点：

1. 依然存在跨库无法join等问题，同时涉及分布式事务，数据一致性问题。
2. 导致多次查询，而且有时候要跨库操作，甚至导致join无法使用，对排序分组等有性能影响
3. 无法支持复杂查询。
4. 事务一致性问题，分布式事务（需要"XA协议"和"两阶段提交"解决），最终一致性（事务补偿）

序列化

- 序列化：把对象转换为字节序列的过程称为对象的序列化。（OutputStream，ObjectOutputStream，objectOutputStream.writeObject(obj)）
- 反序列化：把字节序列恢复为对象的过程称为对象的反序列化。

序列化：将对象状态信息转化成可以存储或传输的形式过程（Java中就是将对象转化成字节序列byte[]的过程）

反序列化：从存储文件中恢复对象的过程（Java中就是通过字节序列转化成对象的过程）

只要我们对内存中的对象进行持久化或网络传输，当我们需要将内存中的对象持久化到磁盘、数据库中时，当我们需要与浏览器进行交互时，当我们需要实现RPC时，这个时候就需要序列化和反序列化了。（JSON格式实际上就是将一个对象转化为字符串，String类型实现了Serializable接口，并显示指定serialVersionUID的值）

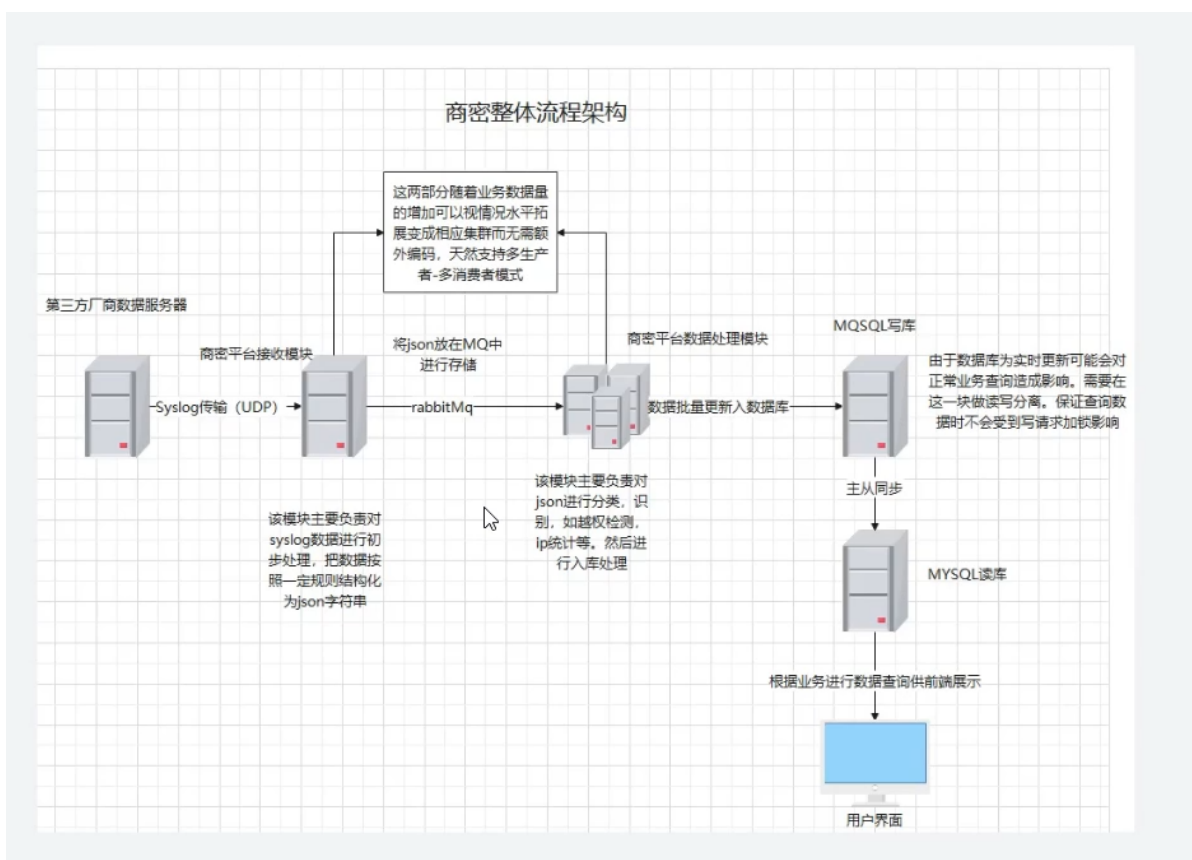
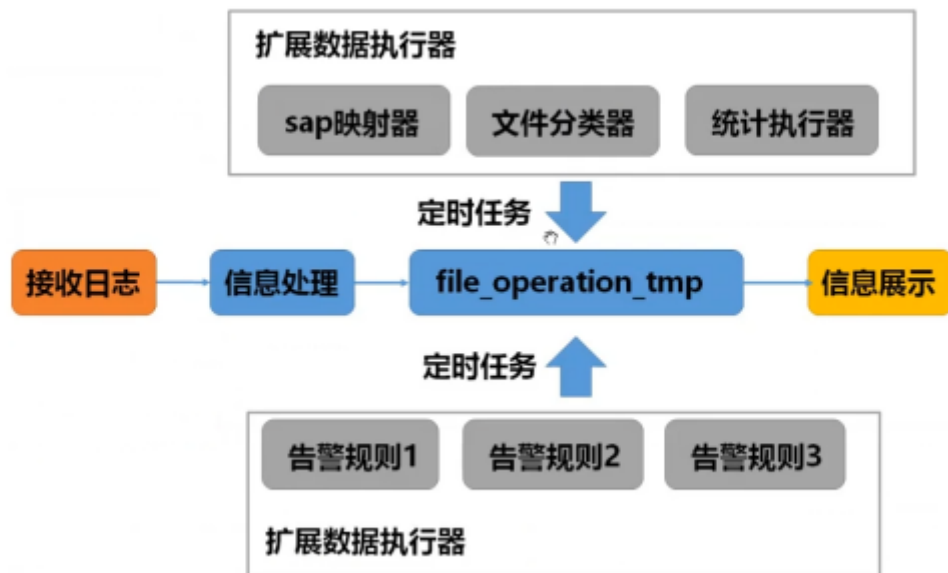
如果不显示指定serialVersionUID，JVM在序列化时会根据属性自动生成一个serialVersionUID，然后与属性一起序列化，再进行持久化或网络传输。在反序列化时，JVM会再根据属性自动生成一个新版serialVersionUID，然后将这个新版serialVersionUID与序列化时生成的旧版serialVersionUID进行比较，如果相同则反序列化成功，否则报错。在实际开发中，不显示指定serialVersionUID的情况会导致什么问题？如果我们的类写完后再不再修改，那当然不会有问题，但这在实际开发中是不可能的，我们的类会不断迭代，一旦类被修改了，那旧对象反序列化就会报错，所以在实际开发中，我们都会显示指定一个serialVersionUID，值是多少无所谓，只要不变就行。

接口响应时间优化（批量接口，定时任务批量请求（如果过少，攒着，过多在拆分），响应数据分发，分布式ID唯一性，例如UUID，或者redis）

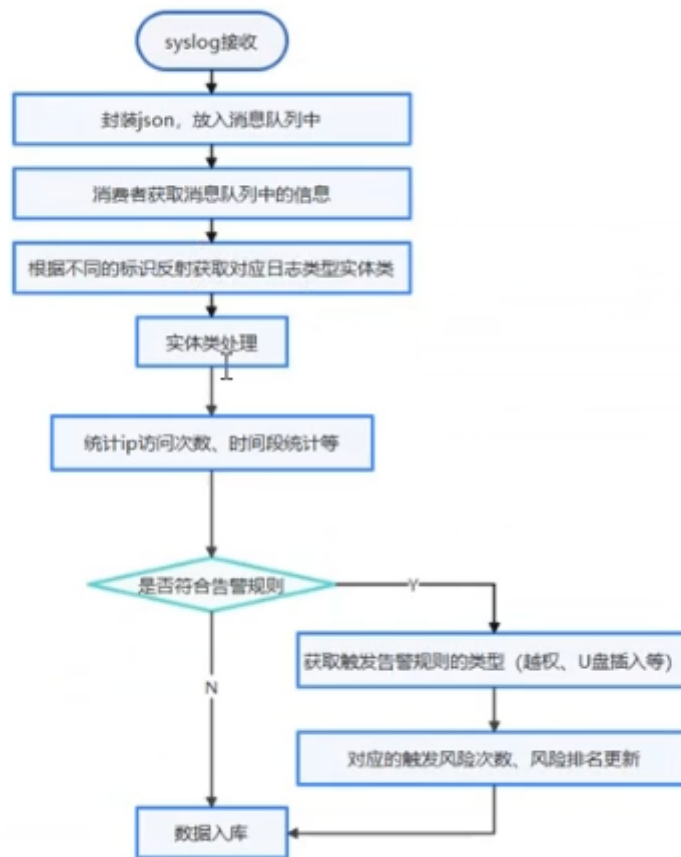
多线程不保证事务，加锁保证，失败事后补偿

● 商密系统技术架构





采集模块：ftp、http、离线上传、syslog接收传输数据（几个G），接收后序列化解析传给mq（缓冲序列化json数据，直接处理压力很大）



当前面的步骤没问题后，代表着数据已经正常入库。这时候需要调用异步任务执行数据分析处理。如下图，scanner和线程池组成了另一个消费-生产模型。需要注意的是，为了确保数据不被重复消费，永远只能有一个scanner（批处理数据）进行数据库扫描（置位需要时间，如果置位的过程中存在另一个 scanner，会导致某条数据会被拿出来消费两次进而导致数据计算不准确）。



ES 是一个**开源的高扩展的分布式全文搜索引擎**

Syslog

系统日志，可以记录在本地，也可以通过UDP协议进行网络传输，端口号514，

作用：

如果一个企业部署了很多系统，**每个系统都有自己的日志模块**，每个系统都是独立的。那么管理员要想通过日志获取到系统的运行信息或者审计日志，那么工作量是相当大的。这个时候，只有要求**所有的系统都将日志传输到syslog服务器上，通过syslog服务器统一管理日志**，就会方便很多。当然这些系统包括所有的硬件，软件系统，他们都会源源不断的发来信息，由**syslog服务器将信息分类，分级别记录**。

spring 封装了 RedisTemplate 对象来进行对Redis的各种操作、异常处理及序列化，支持发布订阅，**支持所有的Redis原生的 API**

RedisTemplate默认采用的是**JDK的序列化策略**，保存的key和value都是采用此策略序列化保存的，查看到的是**字节数组**

个人介绍

目前在一家公司当java实习生，因为目前是秋招，所以想找一份java开发相关的工作，我在这家公司当了差不多4个月的java开发实习生，对应这个开发工作基本熟悉，每周也是按时完成需求开发并进行测试，基本上是独立完成的，偶尔会遇到一些奇怪的bug就会面向百度编程，或者想其他大佬询问，

你们对于这个岗位的一个要求是什么，hc有多少，

我感觉这次面试好像挂了，对于我的面试评价，有哪些需要查漏补缺的
笔试占比高吗

简历

- 熟练掌握 JavaSE 相关知识，具备面向对象的设计思想，对集合、线程、锁等有深入理解
- 熟悉 JVM 原理，类加载机制，掌握常用 GC 算法，熟悉常用的垃圾回收器
- 熟悉并发编程，对 Java 的锁机制，Synchronized、CAS、AQS 等原理有深入研究
- 熟悉 Spring、MyBatis、SpringBoot、SpringCloud 等开发框架，拥有丰富的实战经验
- 熟悉 SpringCloud 微服务技术，熟悉 Nacos、OpenFeign、Gateway、Sentinel 等组件的使用
- 精通 SQL 语言和 MySQL 调优，掌握索引底层原理，事务和锁机制，并具备线上项目调优经验
- 精通 Redis，掌握数据持久化机制，分布式锁，主从和哨兵机制，在项目中多次使用
- 熟悉使用 RocketMQ 消息中间件，在项目中熟练使用
- 熟悉使用 Elasticsearch 进行数据搜索，掌握 ES 倒排索引、分词、集群等
- 熟练使用 Git、Maven、IDEA 等开发工具，拥有良好的编码能力

- 熟练掌握 Java，具备良好的面向对象编程思想。熟悉多线程、集合。
- 熟练使用 Spring Framework、Springboot、Mybatis 等开源技术框架。
- 熟练 MySQL 数据库以及 Redis 非关系型数据库，并且了解 MySQL 的优化。
- 熟练掌握 IDEA、Maven、Git 等项目开发及管理工具。
- 熟悉 SpringSecurity 等权限技术栈以及 SpringCloud 微服务框架。
- 熟悉 Dubbo、ZooKeeper 分别实现表现层和业务层解耦。
- 熟悉 Nginx、Tomcat 服务器的使用。
- 熟悉 RabbitMQ 消息中间件的使用。
- 熟悉 Linux 的常用命令。
- 熟悉 JVM 及常用设计模式。
- 了解 Javascript 编程、jQuery、AJAX、Vue 以及 ElementUI 等前台技术。

简历修改

1、个人情况

姓名，性别，邮箱，联系方式，学校，专业（好的写，不好的不写），籍贯

主要注意格式

2、技能描述，自我评价（工作经验较多）

0、精通/熟悉各种数据结构，对线性表，树、图等都有深入理解，并在leetcode有500题的储备

1、精通/熟悉java核心，多年一线研发经验，具备良好的编程能力，并熟练应用设计模式

熟练掌握javase基础知识，对集合、线程、io都有深入理解，具备良好的面向对象的编程思想，并熟练掌握设计模式

2、精通/熟悉java并发编程，对java的各种锁机制，线程池机制，AQS，CAS，ThreadLocal等都有深入理解，并在项目中熟练使用

3、精通/熟悉JVM，对GC算法和各种垃圾回收器有深入了解，熟悉JVM调优，有实战经验

4、精通/熟悉spring，springmvc，mybatis，springboot等核心框架，多年实战经验，可以根据需求快速搭建项目，看过spring的IOC实现原理的核心源码，看过springboot自动装配的核心源码

5、精通/熟悉springcloud，springcloudalibaba，dubbo，对（nacos，sentinel）各种组件有深入理解，对分布式的服务划分，服务治理，服务分层等有深入理解，并具备线上项目经验

6、精通/熟悉zookeeper底层原理和实现机制，了解paxos，zab算法，对CAP有深入理解

7、精通/熟悉sql语句及mysql优化，对mysql的事务、锁、索引、存储引擎等都有深入研究，并参与过线上的调优工作，分库分表

8、精通/熟悉redis，了解底层磁盘及io模型，数据持久化机制，对于redis的单机和集群模式都有深入理解

9、精通/熟悉kafka，看过核心原理，对底层刷盘机制，集群分片机制有深入了解，并对消息丢失和消息重复有对应的线上解决方案

10、es

11、http，https，网络

12、操作系统、io

13、nginx，lvs，lua

专业技能

- 精通 JAVASE，多线程并发编程，网络编程，锁机制，IO 模型，具备良好的编码能力，熟悉常见设计模式；
- 熟悉 JVM 原理、JMM 模型，常见垃圾回收算法、垃圾回收器及 JVM 调优；
- 熟悉 SSM，Spring，Spring Boot 技术栈；
- 熟练掌握 Dubbo，Spring Cloud 等微服务框架组合构建分布式微服务系统；
- 熟悉 SQL 语言与 MYSQL 调优，对索引优化及存储引擎有深入理解，熟悉 MYSQL 高可用集群搭建，熟悉 Mycat、ShardingSphere 做分库分表；
- 熟悉常用的缓存中间件 Redis，Memcached，Encache，对 Redis 数据持久化机制、多数据类型缓存应用、高可用机制以及多种分布式集群实现有深入理解；
- 熟练掌握 Kafka，RocketMQ 等消息中间件及集群部署；
- 熟悉使用 Elasticsearch 搜索中间件进行数据搜索，能使用 Elasticsearch Stack 搭建 ELK 日志收集系统；
- 熟悉 Zookeeper 分布式协调框架使用，了解其底层原理、实现机制，了解 Paxos、Zab 算法；
- 熟悉基于 Redis，Zookeeper 实现的分布式锁方案；
- 熟悉分布式事务 2PC、TCC，可靠消息最终一致性、最大努力通知方案；
- 熟悉基于常用的 CAP，BASE，AKF 理论进行分布式架构设计；
- 熟悉大型网站高并发架构，熟悉 Nginx、Lvs、keepalived、CDN、高性能网关等高并发、高可用、高吞吐架构方案；
- 了解互联网安全有一定了解，掌握对互联网 API 接口安全，幂等主流设计方案；
- 熟练常用的用具：PowerDesigner，Git，Svn，Maven；
- 熟练使用 Jmeter，AB 进行项目的压测与性能调优；
- 了解掌握基于 Jenkins，Docker，k8s（Rancher）搭建自动化部署服务集群
- 了解大数据相关技术栈，基于 Flume+Hadoop+Hive+HBase+Mysql+Sqoop 做过一些简单日志分析系统 Demo

- 7、精通/熟悉sql语句及mysql优化，对mysql的事务、锁、索引、存储引擎等都有深入研究，并参与过线上的调优工作，分库分表
- 8、精通/熟悉redis，了解底层磁盘及io模型，数据持久化机制，对于redis的单机和集群模式都有深入理解
- 9、精通/熟悉kafka，看过核心原理，对底层刷盘机制，集群分片机制有深入了解，并对消息丢失和消息重复有对应的线上解决方案
- 10、es
- 11、http，https，网络
- 12、操作系统、io
- 13、nginx，lvs，lua
- 14、docker，k8s
- 15、mongodb

3、项目（项目价值，个人价值）

项目名称：偏生产环境 XXX办公自动化平台

项目描述：三四行文字描述即可，切忌打断文字堆积，要凸显出重点，有什么核心价值（具体的数据量），项目解决了什么问题（提供了什么行业的那些具体的解决方案，有多少企业或个人在使用项目），可以适当加业务模块，但是不要全部是业务模块，

技术架构：单纯罗列技术名词即可 144115214146701081

职责描述：一定要分条展示，同时要虚（需求分析，数据库建模，代码review）实（实际的业务开发点）结合，这块至少要写5条以上，凸显在项目中的核心价值

项目难点：一般项目是没有难点的，编造难点（调优、重构、架构设计），你自己的项目很多细节点自己都不清楚，面试官不可能对你的项目细节有足够的了解和认知，所以大胆吹牛就行了，如果面试官提出了质疑，先认可他说的是对的，同时反思自己没有考虑这么多，下去之后跟公司领导反馈下

3、项目（项目价值，个人价值）

项目名称：偏生产环境 XXX办公自动化平台

项目描述：三四行文字描述即可，切忌打断文字堆积，要凸显出重点，有什么核心价值《具体的数据量》，项目解决了什么问题（提供了什么行业的那些具体的解决方案，有多少企业或个人在使用项目），可以适当加业务模块，但是不要全部是业务模块，

技术架构：单纯罗列技术名词即可

职责描述：一定要分条展示，同时要虚（需求分析，数据库建模，代码review）实（实际的业务开发点）结合，这块至少要写5条以上，凸显在项目中的核心价值

项目难点：一般项目是没有难点的，编造难点（调优、重构、架构设计），你自己的项目很多细节点自己都不清楚，面试官不可能对你的项目细节有足够的了解和认知，所以大胆吹牛就行了，如果面试官提出了质疑，先认可他说的是对的，同时反思自己没有考虑这么多，下去之后跟公司领导反馈下

4、自我评价

个人不建议分条展示，写成一个段落即可，展示的是行业的经验和技术的背景，可以从业务背景，技术深度，解决方案等多个维度开始聊，比如做过哪些行业的项目，提出过什么解决方案，解决了什么问题，接触过多大数据量的项目

对于经验不多的同学注意，此处要说明自己的学习能力和技术储备能力

对技术有热情，深入研究过XXXX，XXXX等技术，对什么技术有深入的理解，leetcode有多少的储备量，项目中获得了哪些成长，自己私下会怎么学习技术，看过哪些核心的技术类的书籍，看过哪些github的项目，做过哪些博客的积累